OPERATING SYSTEMS

操作系统入门

从基础 C/C++ 出发

逐步理解程序、内存与内核

基础详解



概念 · 源码 · 实验

19 CHAPTERS

OPERATING SYSTEMS / XV6

前言

你可能已经会写循环、函数、数组和结构体，也写过 `printf("hello\n")`。但如果继续追问：这几个字符是谁送到屏幕上的？程序为什么可以同时运行？一个指针到底指向什么？程序退出以后，内存为什么还能给别人使用？只掌握语言语法，确实很难立即回答。

这正是本书的起点。我们先认识一台计算机怎样执行程序，再讨论为什么需要操作系统。进程、内核、虚拟地址、系统调用、中断等词，会在具体问题中逐个出现。每次遇到新词，先解释它表示什么，再看它解决什么问题，最后才接触实现中的结构体和函数。

全书以 xv6 为实践对象。xv6 是一个用于教学的小型 Unix 风格操作系统，你可以把它理解为一间设备齐全、规模较小的实验室。在这里，进程如何诞生，地址如何翻译，文件如何写入磁盘，都能沿着真实代码找到答案。第一次阅读时，理解现象和机制比记忆源码名称更重要。

请允许自己分几次理解同一件事。第一次知道“进程可以暂停”，第二次弄明白“暂停时要保存寄存器和栈”，第三次才看懂“内核怎样用上下文切换恢复它”。这种层层展开的学习方式，能够把语言语法与机器行为真正连接起来。

本书不会要求你先熟悉汇编、计算机组成原理、Linux 命令或数据结构课程。必要的知识会在使用前补上。你需要准备的是纸笔、一个可以运行示例的环境，以及愿意把一句“我懂了”变成一个可解释例子的耐心。

读完之后，希望你能用自己的话讲清楚：程序怎样得到 CPU，为什么只能访问自己的内存，怎样请求内核服务，等待设备时发生什么，以及一个文件为什么能在程序结束后继续存在。随后再看源码，函数名便会成为已有理解的坐标。

OPERATING SYSTEMS / XV6

怎样使用这本书

从你已经会的知识出发

本书假定你会声明变量、编写函数、使用 if 和循环，见过数组、指针、结构体。即使指针还不熟练，也可以从第 1 章读起，第 4 章会重新解释它与内存的关系。全书使用 C 风格代码；C++ 读者需要留意 xv6 没有 C++ 标准库，也不使用类、异常和 STL 容器。

前 3 章建立机器、程序和操作系统的基本图景，第 4 章补齐读源码的方法，第 5 章才准备 xv6 环境。第 6 章之后，每次围绕一个问题深入一个机制。安装环境暂时有困难时，可以先完成纸笔推演，继续建立概念。

学习阶段	章节	先能回答的问题
认识运行中的计算机	1至3	CPU 在做什么？程序如何运行？操作系统负责什么？
把语言连接到机器	4至6	指针和栈是什么？如何启动 xv6？内核怎样开始运行？
理解内存与执行	7至10	地址为什么要翻译？进程如何切换？用户怎样进入内核？
理解共享与通信	11至13	系统调用如何接线？锁保护什么？管道为什么会等待？
理解文件与设备	14至17	名字如何找到数据？掉电后怎么恢复？shell 怎样启动命令？
独立验证理解	18至19	如何定位错误？如何完成一个范围清楚的小项目？

每章都按同一种节奏学习

先读一个现象或问题，尝试预测结果。接着阅读定义和步骤，使用表格或图检查自己的预测。再看小段代码，把每个变量对应到一个具体对象。最后完成自测：先遮住参考思路，用自己的话回答，再比较遗漏在哪里。

一张纸上最好同时记三件事：对象是什么，它当前处于什么状态，下一步由谁改变它。例如分析文件读取时，先写“进程等待磁盘”，再写“设备完成后通知内核”，最后补上函数名。不要一开始只抄一串陌生名称。

代码、术语和进阶内容的约定

“宿主机 C 示例”可以在普通 C 开发环境中运行；“xv6 用户程序”使用 xv6 的头文件与接口，需要放入它的构建系统；“伪代码”省略了实现细节，只用于推演，不能当作完整实现直接复制。汇编片段通常只展示一小段状态变化。

术语卡会给出新词的直接含义，正文再解释边界。例如“内存页是一块固定大小的内存单位”，还要继续理解“页表也需要占内存”“页和磁盘块不是同一种东西”。附录的术语表用于回查，第一次阅读仍以正文里的例子为主。

标为“进阶”的部分允许稍后再读。先掌握基本机制，再研究严格权限、特殊地址和并发细节，会更容易把握它们为什么存在。完成一章的标准是能解释一条具体路径，而非背出全部英文缩写。

实验环境与版本

本书采用 MIT 2024 年版 xv6 教材所描述的经典 RISC-V 机制作为基础路径。源码持续演进，不同课程分支的名称或实现位置可能变化。第 5 章教你记录版本，附录 B 集中解释差异。基础正文把懒分配、写时复制等扩展与经典基础实现分别说明。

OPERATING SYSTEMS / XV6

目录

点击章名跳转；全部节标题可在 PDF 书签中展开。

前言 . . . 2
怎样使用这本书 . . . 3
第 1 章 计算机怎样一步步执行指令 . . . 6
第 2 章 从源代码到运行中的程序 . . . 10
第 3 章 为什么需要操作系统 . . . 15
第 4 章 把 C 语法连接到内存和机器 . . . 19
第 5 章 准备 xv6，辨清每一层环境 . . . 23
第 6 章 内核怎样从启动入口开始工作 . . . 27
第 7 章 虚拟内存：同一个地址为什么互不干扰 . . . 31
第 8 章 内存从哪里来，又怎样归还 . . . 36
第 9 章 进程怎样暂停、切换和退出 . . . 41
第 10 章 陷阱：CPU 怎样把控制权交给内核 . . . 46
第 11 章 系统调用：把用户请求接到内核服务 . . . 50
第 12 章 并发与锁：一起使用资源为什么会出错 . . . 54
第 13 章 文件描述符与管道：把程序连接起来 . . . 59
第 14 章 文件系统：名字怎样找到磁盘上的字节 . . . 64
第 15 章 写了一半突然断电：日志怎样恢复一致 . . . 69
第 16 章 设备驱动：内核怎样等到硬件完成 . . . 73
第 17 章 shell 与 exec：一行命令怎样真正运行 . . . 77
第 18 章 调试：让每一次观察回答一个问题 . . . 81
第 19 章 循序完成一个小型操作系统项目 . . . 85
结语 从会写语句，到理解运行中的系统 . . . 89
附录 A 按学习顺序查术语 . . . 90
附录 B 源码导航与版本差异 . . . 93
附录 C 实验记录与进阶阅读 . . . 95

第一篇 / 先认识程序与计算机

01 计算机怎样一步步执行指令

学习目标：用简单例子区分 CPU、寄存器、内存和磁盘；理解指令、地址、字节与程序计数器。

1.1 从 x = x + 1 开始

假设你写了 `int x = 3; x = x + 1;`。从 C 的角度看，结果是 x 变成 4。从机器的角度看，需要某个地方保存 3，需要执行加法，还需要把结果保存起来。这个例子已经包含计算机最基本的两类工作：保存数据和按规则处理数据。

CPU 是中央处理器，负责执行机器指令。内存保存正在使用的程序指令和数据。磁盘或固态硬盘保存文件，通常在断电后仍保留内容。寄存器是 CPU 内部数量较少、访问很快的存储位置，用于当前运算和执行状态。

可以暂时把内存当作工作台，把磁盘当作文件柜，把寄存器当作手边的小格子。这个比喻只帮助理解用途；机器真正依靠地址、指令和电路工作，并没有一个人在理解你的 C 源码。

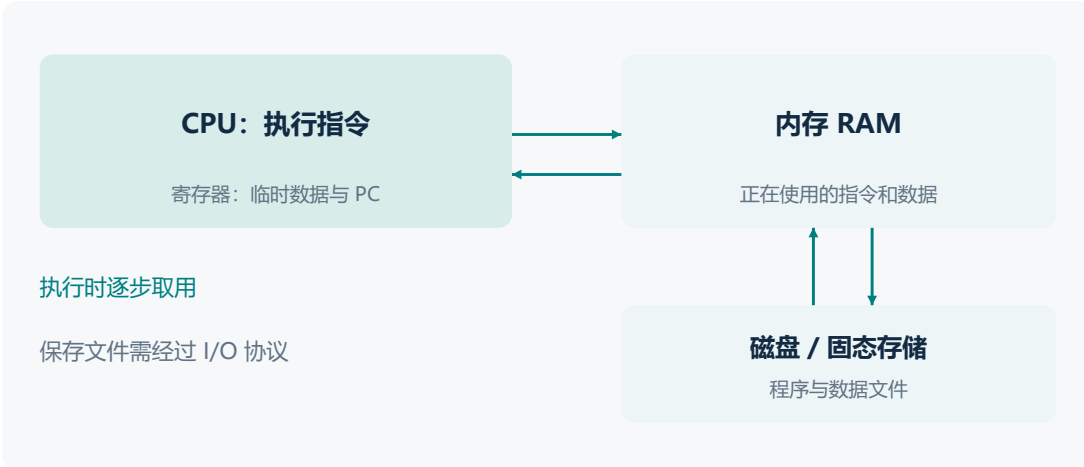


图 1-1 计算、当前状态、工作内容与持久文件各有位置

1.2 什么是字节，地址在给谁编号

计算机用二进制位保存信息，一位只能取 0 或 1，英文叫 bit。8 位组成一个字节，英文叫 byte。字节可以表示字符的一部分、整数的一部分或一段指令，含义由使用方式决定。

为了找到数据，内存中的位置有编号，这个编号叫地址。可以先想象有一排小格，每格保存 1 字节，地址依次是 1000、1001、1002。实际计算机的地址远多于这个小例子。

若一个整数占 4 字节，并从地址 1000 开始，它会占用 1000 到 1003 四个位置。具体类型大小取决于平台和类型，可以用 `sizeof` 检查。地址告诉你“从哪里开始”，类型与长度帮助你确定“怎样解释以及读多少”。

> 术语卡：地址是位置编号，值是该位置保存的内容。地址 1000 和整数 1000 可以有相同的数字写法，却表达不同的含义。

常见容量单位中，1 KiB = 1024 字节，1 MiB = 1024 KiB。本书计算内存页时使用这些二进制单位。十六进制以 `0x` 开头，例如 `0x10` 等于十进制 16；每一位十六进制数对应 4 个二进制位，因此适合阅读地址和权限位。

1.3 CPU 并不直接执行 C 语句

编译器会把 C 程序转换成目标机器能识别的指令。机器指令可以表达“把数据从内存读到寄存器”“两个寄存器相加”“把结果写回内存”“改变下一条执行位置”等动作。

下面是教学用的指令步骤，没有采用某个真实汇编器的完整语法：

伪代码 / 只用于机制推演

```
R1 = 从保存 x 的位置取值     // R1 变为 3
R1 = R1 + 1                 // R1 变为 4
把 R1 写回保存 x 的位置      // x 变为 4
```

实际编译器可能优化代码，把变量一直保留在寄存器，或直接计算出结果。学习操作系统时，先用这个简化过程理解“读取、计算、写回”是不同动作；以后分析并发时，这个区别会非常关键。

一条 C 语句可能对应多条指令，几行 C 也可能被合并成更少指令。源码行数不能直接代表 CPU 执行次数，更不能据此认定一行操作不会被打断。

1.4 下一条指令由谁决定

CPU 有一种用于记录执行位置的状态，通常称为程序计数器，英文 program counter，简称 PC。它指示从哪里取得下一条要执行的指令。顺序执行时 PC 向后移动；遇到条件跳转、函数调用或返回时，执行位置按指令规则改变。

想象一段程序先做加法，再检查是否大于 10，最后决定进入哪个分支。CPU 不会一次看懂整个 if，而是一条一条执行比较与跳转指令。所谓“程序正在某处运行”，最终要落实到某个执行位置和一组寄存器状态。

如果想让程序暂停后继续，仅保存变量还不够。还需要知道停在什么位置、临时运算结果在哪里、当前函数用了哪段栈。第 9 章的进程切换，就是系统地保存和恢复这些状态。

1.5 内存和磁盘为什么不能混为一谈

程序文件可以长期保存在磁盘上。运行时，系统为它建立内存中的执行环境；CPU 主要通过这个环境取得指令与数据。程序运行中修改了一个变量，不代表已经修改了磁盘文件。

例如你用程序读入一个文本，在数组里把某个字符改掉，然后立即退出。如果没有通过文件写入接口保存，新内容一般不会出现在原文件中。内存中的数组与磁盘中的文件是两份相关但独立的状态。

内存通常比持久存储更适合频繁访问，但普通 RAM 断电后不能保留内容。把数据放入内存与把数据可靠写入磁盘，是不同任务。第 15 章会继续解释“已经调用 write”为什么还需要讨论何时真正持久化。

1.6 一颗 CPU 为什么看起来能同时做很多事

假设只有一个 CPU 核，它可以先执行程序 A 一小段，保存 A 的状态，再执行 B，之后继续 A。切换足够频繁时，人会感到两个程序同时在推进。这叫并发：多个任务的执行过程在时间上交错。

如果有多个核，A 与 B 还可能在同一时刻分别由两个核运行，这叫并行。并发不要求真的同时执行，并行强调同一时刻有多个执行者。实际操作系统经常同时使用这两种方式。

切换需要安排者，也需要保存状态的方法。系统还要保证 A 不会覆盖 B 的数据。至此，操作系统要解决的问题已经开始出现，但我们先在下一章把“程序怎样变成可运行对象”讲清楚。

1.7 停下来推演

问题 1：磁盘上有 hello 程序，CPU 是否就一定在运行它？ 不一定。保存文件只说明内容存在，执行还需要建立内存与 CPU 的运行状态。就像保存了一份菜谱，不代表正在做菜。

问题 2：保存变量就能恢复程序吗？ 还不能。程序执行位置、寄存器和函数调用的栈状态也影响接下来该做什么。

纸笔练习。 画出 CPU、寄存器、内存和磁盘四个方框。把“当前加法的临时结果”“源代码文件”“运行中的数组”“下一条指令位置”分别放入适当方框。参考：临时结果常在寄存器，源文件在持久存储，数组通常在内存，PC 是处理器执行状态。

本章先记住：CPU 执行指令，寄存器保存当前少量状态，内存保存工作内容，持久存储保存需要留下的文件。

第一篇 / 先认识程序与计算机

02 从源代码到运行中的程序

学习目标：理解编译、链接、装载与运行；认识代码区、数据、栈和堆；分清函数调用与程序启动。

2.1 写完 main 之后还发生了什么

下面是一个可在普通 C 开发环境运行的小程序。我们把运行编辑器和编译器的电脑叫宿主机，稍后还会用它运行模拟器。

宿主机 C 示例 / 按注释区分完整程序与片段

```c
#include <stdio.h>
int add(int a, int b) {
  return a + b;
}
int main(void) {
  int result = add(2, 3);
  printf("%d\n", result);
  return 0;
}
```

源码是给人和编译器阅读的文本。计算机要执行它，还要得到目标指令和文件布局。常见工具链先处理头文件与宏，再编译产生目标文件，之后链接成可执行文件。你在 IDE 中按“运行”，工具通常帮你完成了其中多个步骤。

术语卡：工具链是一组相互配合的开发工具。编译器负责翻译源代码，链接器连接多个目标文件和需要的符号，调试器帮助观察执行状态。

2.2 编译和链接分别解决什么问题

假设 add 写在一个文件，main 写在另一个文件。编译 main 时，编译器可以根据声明知道 add 怎样接收参数、怎样返回结果，但最终还要找到 add 的实现位置。链接器把各部分连接起来，解决符号引用，并按布局规则组织代码和数据。

符号是函数名、全局变量名等对象在工具链中的标识。例如缺少 add 的实现，就可能出现 undefined reference 一类链接错误。语法正确并不自动保证所有被调用对象都有实现。

`printf` 又增加了一层：它通常来自运行库。库是供多个程序使用的一组代码。包含头文件主要提供声明，并不等于已经把实现复制进程序。静态链接、动态链接的细节各不相同，本书先掌握“声明、实现和连接”这三个角色即可。

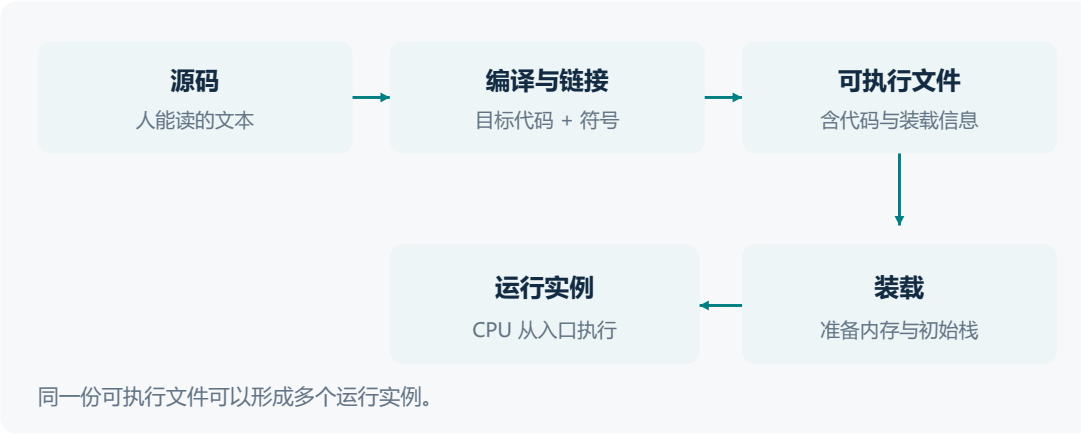


图 2-1 翻译与链接产生文件，装载再建立运行环境

2.3 可执行文件还不是正在运行的实例

可执行文件除了指令，还包含系统装载它所需的信息，例如哪些内容放入内存、入口位置在哪里。装载器根据这些信息准备地址空间、初始栈和必要状态，然后让 CPU 从入口开始。

装载可以理解为“把运行条件准备好”。它通常不是把整个文件从头到尾原样复制到某块内存。文件里的头部、代码段和数据段有各自含义；某些全零区域甚至不需要在文件里保存大量零字节。

在普通 C 环境中，最早的入口常先执行运行库启动代码，再调用 main。xv6 的启动与用户程序也有各自的入口约定。因此，“程序从 main 开始”适合描述你自己编写的主要逻辑，却不足以解释整台机器刚启动时发生什么。

2.4 用一次 add 调用理解栈

main 调用 add 后，必须记得加法结束要回到 main 的哪里。如果函数还需要保存局部信息，也得有一个按调用层次组织的位置。栈就是常用的管理方式：较晚进入的调用，通常较早结束，符合后进先出的顺序。

可以把每次调用留下的一组信息看作一个栈帧，英文 stack frame。它可能包含保存的寄存器、局部变量以及返回所需信息。实际内容由调用约定、编译器和优化决定，并非每个局部变量都必然在栈中。

执行阶段	当前工作	需要保留什么
main 调用 add 前	准备参数 2、3	main 稍后继续的位置与仍需使用的状态
add 执行中	计算 2 + 3	计算结果与返回所需信息
add 返回后	main 获得 5	已结束调用的临时空间可复用

局部变量的地址不能在函数结束后继续当成有效对象使用。栈上的字节可能暂时还在，但其生命周期已经结束，下一次调用可能覆盖它。语言里的“变量作用域”与运行时的“存储生命周期”在这里发生联系。

2.5 全局数据、栈和堆各适合什么

全局变量和静态变量具有较长的存储期。局部自动变量通常随函数调用使用和结束。若程序需要一块大小运行时才确定、并且要跨函数保存的空间，常通过 malloc 等接口取得动态内存，这类空间通常称为堆。

栈和堆都可能位于同一个进程的内存环境中，“堆”不是一种独立硬件。malloc 通常先管理已有空间，需要更多时再向系统申请。free 把一块空间交回分配体系，未必立即让操作系统归还每一页物理内存。

宿主机 C 示例 / 按注释区分完整程序与片段

```c
// 函数片段：只用于观察局部对象与动态对象的寿命。
#include <stdlib.h>
int *make_value(void) {
  int *p = malloc(sizeof(int));
  if (p != NULL) *p = 42;
  return p;
}
```

这里的 stdlib.h 提供 malloc、free 等接口声明。函数返回后，动态对象仍可由调用者使用，并由约定的一方 free。若返回的是局部变量地址，情况就不同，因为函数结束时局部对象的生命周期已经结束。

2.6 同一份程序可以有多个运行实例

你可以打开两个编辑器窗口，也可以执行同一个命令两次。它们的指令可能来自同一个可执行文件，但各自有自己的当前位置、数据和资源连接。操作系统把这样的运行实例组织成进程。

术语卡：程序是一份可执行的描述，进程是这份程序的一次运行及其相关状态。同一个程序可以对应多个进程，一个进程也可以在执行期间换成另一份程序。

后一种情况暂时不必急着理解实现。第 17 章会介绍 exec：保留进程身份，替换当前运行内容。此处只要把“文件”和“运行实例”分开，就已经跨过一个重要门槛。

2.7 停下来推演

问题 1：编译成功但提示找不到某个函数，问题一定在语法吗？ 不一定。目标文件可能缺少实现，或链接步骤没有把所需对象连接进来。

问题 2：为什么函数结束后不能继续用局部变量地址？ 对象的生命周期已经结束，原有地址不再代表你拥有的有效对象。地址数值仍可打印，不等于访问有效。

小练习。 写出 add 程序从保存源码到输出 5 的五个阶段：编译相关处理、链接、准备运行环境、执行 main/add、通过输出设施显示结果。暂时把最后一步的系统机制留作问号，第 3 章开始解释它。

本章先记住：源代码需要翻译和连接；可执行文件需要装载；运行时的函数调用和数据有各自的状态与寿命。

第一篇 / 先认识程序与计算机

03 为什么需要操作系统

学习目标：理解资源管理、内核、用户程序、隔离与接口；从日常现象认识进程、系统调用和中断。

3.1 如果几个程序都直接控制硬件

设想两个程序都认为内存地址 1000 属于自己。A 刚把成绩写到那里，B 就把同一位置当作图像缓冲。又设想一个程序永远不主动停下，独占 CPU；另一个程序随意重写磁盘上保存目录信息的区域。即使每个程序语法都正确，整个系统也很难可靠工作。

操作系统要管理这些公共资源，并建立程序之间的边界。它决定谁可以运行、哪一部分内存可以访问、如何打开文件以及怎样与设备交互。程序因此能使用一组相对统一的服务，而不必每个都独立控制硬件。

操作系统通常还包括很多用户态工具。本书聚焦其中最核心的部分：内核。内核在具有较高权限的运行环境中管理资源，普通应用主要在受限制的用户环境中执行。

3.2 内核不是一个普通函数库

函数库和你的程序通常在同一个执行环境里工作。内核则位于受保护的边界另一侧。用户程序可以调用自己的加法函数，却不能因为知道内核函数名，就直接随意修改内核的进程表或页表。

这种边界需要硬件配合。CPU 的特权模式限制某些指令与状态的使用，内存保护限制可访问的位置。普通程序即使出现坏指针，也应主要影响自己，而不是把整个系统的管理数据写坏。

术语卡：用户态是普通程序使用的受限执行状态；内核态是内核处理服务和事件时使用的受保护执行状态。它们描述 CPU 的执行权限，不能简单等同于磁盘上的两个目录。

在 RISC-V 中，本书后面会用 U 模式表示用户态，用 S 模式表示内核主体所在的监督者模式。先记住权限不同即可，具体寄存器和切换过程放到第 6、10 章再读。

3.3 程序怎样请求打印字符

回到 `printf("hello\n")`。printf 主要负责把数据格式化成字符，输出还要经过环境提供的接口。Unix 风格系统中，底层经常以 write 一类系统调用把字节交给内核管理的输出对象。

系统调用是用户程序请求内核服务的受控入口。用户说明“做什么”和“参数是什么”，内核检查请求，再完成允许的工作。它有明确的入口与返回规则，与同一权限内的普通函数调用不同。

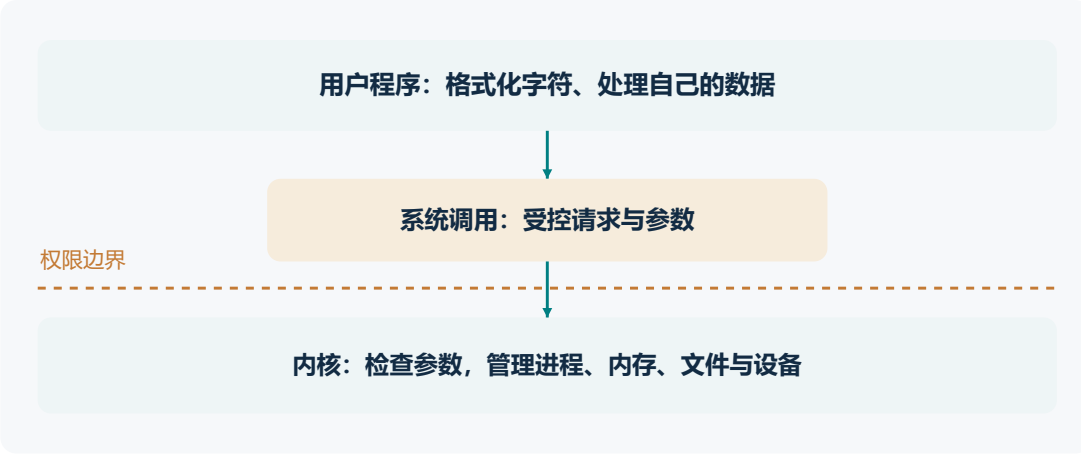


图 3-1 普通程序通过受控接口使用内核管理的资源

例如要写 6 个字节，内核不能只相信用户传来的地址。那个数值可能指向不存在的页面或受保护区域。内核需要验证整个访问范围，并检查输出对象是否有效。错误的普通请求应返回失败或结束当前进程，不能轻易让整个系统崩溃。

3.4 进程让多个程序有各自的运行现场

假设 A 已经算出中间值 7，接下来要乘 2；B 正在读取文件。操作系统切换时，必须保留 A 的执行位置与中间状态，稍后继续得到正确结果。进程就是组织这些状态与资源的重要对象。

每个进程有自己的身份编号，常称 PID，即 process identifier。PID 用于区分运行实例，但编号本身不包含完整状态。内核还保存它的内存环境、打开的文件和是否正在等待等信息。

负责选择接下来运行哪个进程的机制叫调度。处在等待键盘或磁盘状态的进程不必一直占着 CPU，调度器可以运行其他准备好的进程。一个进程在等待，不代表整台机器都必须等待。

3.5 为什么需要虚拟地址

给程序一套自己的地址编号，可以简化程序布局并保护其他进程。A 和 B 都使用地址 `0x1000`，但经过各自的映射后到达不同物理内存，就不会因为编号相同自动冲突。

程序使用的这类地址叫虚拟地址；真正对应 RAM 或设备位置的地址叫物理地址。规定转换关系的结构叫页表。你现在只需理解“编号需要经过一张映射表”，暂时不必记住三级索引和权限位。

页表可以同时规定允许读、写或执行。有些地址根本没有映射。这样，错误指针可以被硬件发现，并交给内核处理。第 7 章会用具体数字一步步完成翻译。

3.6 中断让设备有机会通知 CPU

读磁盘需要时间。如果程序反复询问“好了没有”，CPU 会把许多时间花在检查上。另一种方式是先提交请求，等设备完成后再通知处理器。这类异步通知称为中断。

“异步”表示事件的到来不必由当前程序执行到某条特定请求指令直接决定。例如时钟到期可以打断正在计算的程序，键盘输入可以发生在任意时刻。设备中断、时钟中断由内核相应路径处理。

异常则通常和当前正在执行的指令有关，例如访问未映射地址。系统调用也是通过特定异常进入内核的一种使用方式。第 10 章会把它们放到统一的 trap 机制里比较。现在先区分“程序主动请求服务”和“设备通知完成”这两个现象。

3.7 文件和设备为什么需要统一接口

普通文件保存一串字节及其相关信息，通常能在程序退出后继续存在。键盘和串口也提供字节输入输出，但不因此成为磁盘上的普通文件。Unix 风格接口用“打开对象后得到编号，再 read/write”组织多种 I/O。

I/O 是 input/output 的缩写，意为输入与输出。一个程序读字节，不必总知道它们来自磁盘文件、键盘还是另一个进程。接口的一致性允许 shell 把程序连接起来，形成管道和重定向。

这里的“文件接口统一”不表示所有设备都具有普通文件的全部行为。例如管道不能像持久文件一样保存到下次开机，串口也没有与普通文件完全相同的定位能力。

3.8 停下来推演

问题 1：为什么普通程序不能随意改页表？ 页表参与规定访问权限。若任何程序都能任意重写它，程序间隔离就失去基础。

问题 2：读取磁盘时为什么可以运行别的程序？ 请求交给设备后，等待进程暂时不具备继续条件。内核保存它的状态，选择别的可运行进程，完成通知到来后再让等待者恢复。

问题 3：系统调用和普通函数调用最重要的区别是什么？ 系统调用跨过权限边界，必须按受控规则进入内核、验证参数并返回。普通函数调用通常在同一个权限和地址环境中进行。

练习。 为“打开文本、读取内容、显示字符”三个动作各写一句内核可能要做的检查。参考：名字是否存在；缓冲地址是否有效；输出对象是否允许写入。接下来的章节会把这些直觉变成实际机制。

第二篇 / 从语言走向系统

04 把 C 语法连接到内存和机器

学习目标：重新理解指针、结构体、位运算和函数指针；认识调用约定，学会分层阅读代码。

4.1 指针保存地址，解引用才访问对象

在 C 中，`int *p` 表示 p 能保存某个 int 对象的地址。`&x` 取得 x 的地址，`*p` 沿该地址访问对象。可以把地址当作门牌号，但门牌号不等于房间里的内容。

宿主机 C 示例 / 按注释区分完整程序与片段

```c
// 可放进普通 C 程序的 main 中观察。
int x = 7;
int *p = &x;
*p = 9;
// 此时 x == 9, p 保存的仍是 x 的地址。
```

执行 `*p = 9` 改变的是 p 指向的对象，p 自己的地址数值没有因此改变。执行 `p = &y` 才会把 p 改为指向另一个对象。这两种变化在跟踪内核链表时必须分清。

合法访问还要求对象存在、类型和范围适合，并且当前环境允许访问。内核中的 `char *` 可能是内核缓冲区，也可能来自用户给出的地址数值。C 的类型名字没有完整表达这些差别，使用者必须另外检查。

4.2 数组边界和指针运算

若 `int a[4]`，有效下标是 0 到 3。指针 `a + 1` 前进的是一个 int 的大小，而非固定 1 字节；字节级位置通常用 char 指针或整数地址计算表达。

可以构造数组末尾后一位置用于比较，但不能把它作为现存元素解引用。内核经常使用固定长度数组管理进程或文件，遍历边界就是资源上限的一部分。

缓冲区参数还需要长度。例如地址有效，但复制 100 字节跨出了对象，仍然可能出错。后面的用户内存复制不仅检查起点，还要处理长度、跨页和算术溢出。

4.3 结构体既是数据集合，也是对象状态

结构体把相关字段放在一起。`p->state` 等价于 `(*p).state`。假设一个简单任务包含编号与状态，内核就可以通过指针找到它并修改当前状态。

C / 机制片段

```c
// 教学用简化对象，并非 xv6 的完整进程结构。
struct task {
  int id;
  int state;
};
struct task t = {1, 0};
struct task *p = &t;
p->state = 1;
```

真实结构体还可能有指向其他对象的指针。指针复制只复制地址，不自动复制地址背后的对象。例如两个 fd 槽位保存同一个 file 指针，得到的是共享对象，而不是两个独立文件实例。

结构体字段之间可能有填充字节，对齐规则也由平台决定。汇编若按固定偏移访问字段，C 的布局必须与之匹配。理解这个联系后，再看 trapframe 中的大量寄存器字段就不会觉得它们凭空出现。

4.4 位运算为什么常出现在内核

一个整数的不同二进制位可以各表示一个开关。例如低三位表示读、写、执行权限，设 R=1、W=2、X=4。`R | W` 得到 3，二进制为 011，表示同时开启读和写。

C / 机制片段

```c
// 教学用权限值，不是实际 PTE 常量定义。
unsigned perm = 1 | 2;  // 011: 可读、可写
int can_write = (perm & 2) != 0;
perm = perm & ~2u;      // 清除写权限
```

按位与 `&` 用来检查或保留某些位，按位或 `|` 用来设置位，取反 `~` 把各位翻转。注意逻辑与 `&&` 与按位与不是同一种运算。页表项后面会同时保存地址信息和权限位，所以阅读时要按字段解释一个整数。

4.5 函数指针是一张可调用的地址表

普通数据指针指向对象，函数指针指向可调用函数。若多个函数具有相同调用类型，可以把它们放进数组，用编号选择。

C / 机制片段

```
// 简化示例：声明之后还需要有相应函数实现。
int action_a(void);
int action_b(void);
int (*actions[])(void) = {action_a, action_b};
// actions[0]() 调用 action_a；调用前要检查索引。
```

这解释了系统调用分发表的直觉：用户提供服务编号，内核检查编号后找到处理函数。查表只是选择入口，参数是否合法仍由相应层检查。第 11 章再看实际 xv6 声明。

4.6 调用约定让 C 与汇编能合作

当一个函数调用另一个函数时，双方需要约定参数放在哪里、返回值放哪里、哪些寄存器必须保留、栈怎样对齐。这组二进制层面的规则叫调用约定，属于 ABI 的一部分。ABI 是应用二进制接口的缩写。

在 RISC-V 中，先认识 `sp` 栈指针、`ra` 返回地址、`a0` 到 `a7` 参数寄存器即可。普通整数返回值常用 a0，某些返回形式还涉及 a1。`s0` 到 `s11` 是被调用者保存寄存器，`t0` 到 `t6` 是临时寄存器。

“被调用者保存”意味着函数如果使用它们，要保证返回时调用者看见应保留的值。“临时”意味着调用者不能指望经过一次调用后仍保持原值。这是上下文切换只保存一组特定寄存器的重要前提。

4.7 读代码时问四个问题

第一，输入是什么，哪个值只是地址？第二，它修改哪个对象，修改由谁看见？第三，新增资源由谁在何时释放？第四，访问时需要什么权限、页表或锁？锁可以先理解为访问共享对象时取得的一种资格：各执行者按规则竞争，取得者用完后归还。第 12 章再系统解释它怎样保证正确，初读先记下源码注释中的限制。

把正确状态中必须始终成立的关系叫不变量。例如“同一块空闲内存不能同时分给两个独占使用者”。源码里的检查和同步，往往就是为了维持这类关系。遇到这个词时，可以先把它读成“这件事绝对不能被弄错”。

`volatile` 要求编译器按相应规则保留访问，常见于设备寄存器。它不能保证两个 CPU 不同时修改数据，也不能替代锁和原子操作。看到一个关键词时，既要知道它提供什么，也要知道它没有提供什么。

读汇编先找保存现场、修改栈、切换地址环境与跳转入口四类动作。大量重复的保存指令可以先看成一个整体，再按需要核对寄存器和偏移。

4.8 停下来推演

问题 1：复制一个结构体指针会复制整份对象吗？ 不会，复制的是地址。是否需要深复制、共享或增加引用，要由程序协议决定。

问题 2：把指针强制转换为另一种类型，能让原本非法的地址变有效吗？ 不能。类型转换不创建物理内存、不延长对象寿命，也不改变页表权限。

练习。 用自己的结构体画一张图：变量 `p` 中保存对象地址，对象内再保存另一个指针。分别修改 `p`、修改 `p->state`、复制 `p`，标出哪一块内存发生变化。能讲清楚这张图，再开始阅读内核的对象关系会轻松很多。

第二篇 / 从语言走向系统

05 准备 xv6，辨清每一层环境

学习目标：知道命令在哪里执行；区分宿主机与目标机器；理解构建产物，并完成一次基础运行。

5.1 为什么用一个小型操作系统学习

xv6 保留了进程、虚拟内存、文件系统和设备驱动等重要机制，同时控制规模，适合把一次请求追踪到底。它运行在 RISC-V 机器上；RISC-V 是一种指令集体系结构，规定处理器能识别哪些指令以及相关行为。

你的电脑可能使用 x86-64 或 ARM，不必因此更换硬件。QEMU 是宿主机上的模拟器程序，可以模拟 RISC-V CPU、内存和设备，让 xv6 在其中运行。模拟器负责呈现硬件行为，xv6 仍要自己管理进程和设备。

5.2 终端、shell、工作目录是什么

终端提供文字输入输出界面，shell 读取并解释命令。你在宿主机终端输入 Make 命令时，执行的是宿主机程序。QEMU 启动后看到 xv6 的 `$` 提示符，此时输入 ls，运行的是 xv6 自己的用户程序。

工作目录是解释相对路径的起点。例如在源码根目录中，`kernel/proc.c` 表示从当前目录进入 kernel，再找到 proc.c。绝对路径从相应系统的根位置开始；宿主机的某个磁盘路径不会自动成为 xv6 文件系统中的路径。

术语卡：宿主机是运行编辑器、编译器和 QEMU 的真实系统；目标机器是工具生成的程序要运行的机器，本书中由 QEMU 模拟。



图 5-1 一台电脑上的四层运行边界

为了避免混淆，实验记录中给命令标注“宿主机”或“xv6”。Make、编译器和 GDB 都在前者运行；cat、ls、echo 等实验命令在后者运行。

5.3 为什么需要交叉编译

交叉编译器在宿主机运行，却为另一种目标环境生成代码。普通 C 编译器默认可能生成你电脑的指令，也可能连接宿主机运行库；这些产物不能直接拿给 xv6 使用。

xv6 的 Makefile 用 `TOOLPREFIX` 等设置选择兼容的 RISC-V GCC（C 编译器）、LD（链接器）和反汇编工具。反汇编是把机器码转换成便于人阅读的指令文字。内核与用户程序各有链接布局；生成镜像的 mkfs 则是宿主机工具，应在宿主机上运行。

Make 是按依赖规则组织构建的工具。若目标文件缺失但源文件和规则存在，它通常会重新生成目标；不能认为删除一个生成文件就一定让构建失败。是否把一个用户命令装进镜像，还要看 `UPROGS` 等列表。

5.4 一次构建得到哪些文件

产物	用来做什么	由谁使用
kernel/kernel	链接后的内核可执行文件	QEMU 启动配置与调试器
user/_cat 等	链接后的目标用户程序	mkfs 打包，xv6 随后装载
fs.img	xv6 磁盘内容的镜像	模拟的 VirtIO 磁盘设备
kernel/kernel.asm	内核反汇编	阅读指令、定位问题
kernel/kernel.sym	内核符号信息	辅助按地址找函数

fs.img 是一个遵循 xv6 文件系统布局的文件，不是宿主机普通目录。宿主机的 mkfs 创建超级块、目录和文件数据等结构。`user/_cat` 打包后在 xv6 中通常叫 cat，用户执行时无需写宿主机路径。

5.5 最小启动步骤

在 Linux 或 WSL 等适合所用源码的环境准备 Make、Perl、兼容的 RISC-V 工具链与 QEMU。各发行版安装包名称和最低版本会变化，应按所用课程或仓库说明配置。WSL 可以理解为 Windows 上用于运行 Linux 环境的一种方式。

在宿主机源码根目录执行下面两条命令。git 的提交号用于记录准确版本；若课程提供的目录不带 Git 信息，就记录课程分支或取得代码的说明。

宿主机命令

```
git rev-parse HEAD
make qemu
```

看到 xv6 提示符后，逐条执行：

xv6 命令与执行记录

```
ls
cat README
echo hello
```

观察目录列表、文本内容与 hello。常见的 QEMU 纯终端配置用 `Ctrl-a`，松开后再按 x 退出。回到宿主机提示符后，才继续执行 Make 等命令。没有必要每次运行都先 make clean，清理应有明确原因。

5.6 从 cat README 记住一条主线

shell 读到命令，创建执行者并装载 cat；cat 打开 README，循环读取字节，再写到标准输出。内核在背后提供进程、内存、文件与设备服务。

这里“装载 cat”和“打开 README”是两个不同任务。前者获得要执行的指令，后者获得命令要处理的数据。它们都可能访问磁盘，但对象和目的不同。

第一次先写这条主线，不要急着展开所有函数。每学完一个子系统，再补一条解释：比如“read 可能等待磁盘”或“write 的目标由 fd 决定”。到第 17 章时，你会有一张完整地图。

5.7 停下来推演

问题 1：在 QEMU 提示符下输入 make 为什么不一定可行？ 那里是 xv6 用户环境，不是宿主机开发环境。xv6 的镜像中通常没有你宿主机的构建工具。

问题 2：宿主机 README 改了，xv6 为什么还可能看到旧内容？ xv6 读的是镜像中的文件。镜像尚未更新或运行时仍使用旧镜像，就不会自动看见宿主机修改。

排错练习。 把“找不到编译器”“未定义符号”“QEMU 不能启动”“xv6 已启动但命令卡住”分别归类为工具、构建、模拟器启动、目标运行问题。先确定层次，再选择观察办法。

源码阅读入口：先只看 Makefile 中的 qemu、UPROGS、fs.img 目标，以及 user/cat.c。暂不要求读懂文件系统与陷阱代码。

第二篇 / 从语言走向系统

06 内核怎样从启动入口开始工作

学习目标：理解启动的先后依赖；认识特权模式和每核状态；跟随控制权走到第一个 shell。

6.1 系统不能一开始就使用自己的全部服务

平时程序能调用 malloc、打开文件，是因为运行环境已经准备好。内核刚启动时，却不能假定自己已经有分配器、进程和文件系统。因此启动不是“调用一个很大的 main 就完事”，而是逐步建立运行条件。

本书讨论的 QEMU 配置把内核放到以物理地址 `0x80000000` 为起点的内存区域，最终把控制权交给 `_entry`。这是具体配置中的内核位置，不是所有计算机通用的上电地址。机器复位后到达内核的过程，还可能经过固件或启动代码。

先把启动想成一串依赖：可用栈使 C 函数能运行；页分配器使内核能取得新内存；页表使地址翻译按预期工作；进程与中断设施使等待设备成为可能；最后才能完成用户环境。



图 6-1 从启动入口到 shell 的权限接力

6.2 多核启动时，每个 hart 都要有自己的栈

hart 是 RISC-V 中硬件执行线程的名称。入门时可以把 xv6 使用的每个 hart 看作一个独立执行者，它有自己的寄存器状态。多个 hart 可以共享内存，但不能因为共享就共用同一条正在变化的启动栈。

假设每段启动栈是 4096 字节，数组起点为 S。hart 0 使用第一段，栈顶设为 S+4096；hart 1 使用第二段，栈顶为 S+8192。栈通常向低地址增长，因此选择每段的高端作为初始 sp。

hart 编号	对应栈区域	初始栈顶
0	S 到 S+4095	S+4096
1	S+4096 到 S+8191	S+8192
2	S+8192 到 S+12287	S+12288

`entry.S` 根据 hart 编号计算这件事，再调用 C 函数 start。若两个执行者都往相同栈位置保存返回信息，一个很容易覆盖另一个，故障可能到很后面才出现。

6.3 先理解权限，再认识 mret

RISC-V 常见的三个特权模式是 M、S、U。M 是机器模式，接近最底层控制；S 是监督者模式，xv6 主体内核在这里工作；U 是用户模式，普通命令在这里运行。硬件限制各模式能执行的操作。

启动中的 start 处于 M 模式。它设置“返回后要到什么模式”和“返回后从哪里执行”，准备把控制权交给 S 模式的 main。保存目标位置的控制寄存器叫 mepc，mstatus 中有关字段规定返回的权限状态。

随后执行 mret，即按 M 模式返回规则恢复特权与执行位置。它与普通函数 ret 有重要区别：普通 ret 主要沿返回地址继续，mret 参与权限状态的交接。S 模式是否能响应具体中断，还取决于相关使能和委托配置，不能只用“mret 打开中断”概括。

术语卡：控制寄存器保存影响 CPU 工作方式的状态，例如陷阱入口、页表根或中断配置。它们与保存普通加法数据的通用寄存器有不同职责。

6.4 main 的初始化顺序为什么不能随意交换

main 先建立控制台和打印设施，使后续故障有机会被看见；准备物理页分配器后，才能给页表等对象分配内存；建立内核页表后，每个 hart 才能安装自己的地址翻译状态。随后准备进程表、陷阱、中断控制器、缓存、文件表和磁盘驱动。

这里的“表”先理解为记录一类对象的管理结构；缓存则先理解为暂存已取得内容、便于复用的结构。启动时先把这些结构准备好，后面的章节再研究它们怎样查找文件、缓存磁盘块。初始化清单不要求你现在就掌握每种对象的内部字段。

需要区分“准备一个表和它的锁”与“真的读磁盘”。例如 inode 表初始化主要准备内存结构；装入文件系统并恢复日志的 fsinit 需要可能阻塞的磁盘 I/O，经典路径将它安排在首次进程的 forkret 中，此时已经有可调度的进程上下文。

这里的“上下文”泛指继续执行所依赖的环境和状态。现在只要理解：某段代码如果可能等设备，就需要有让出 CPU、稍后继续的条件；在最早启动阶段，这些条件尚未建立。

6.5 哪些事情做一次，哪些每个核都做

共享的全局表通常只初始化一次，否则多个 hart 可能同时清空或重建同一对象。每个 hart 自己的页表寄存器、陷阱向量和中断相关配置，则需要各自设置。

因此一个 hart 完成主要全局工作，其他 hart 等待“已经准备好”的标志，再进行自己的配置。标志还必须配合适当的内存顺序保证，让其他执行者看见它时，也看见此前完成的初始化结果。第 12 章会进一步解释可见性。

读源码时把初始化调用写成两栏：“创建共享对象”和“安装本核状态”。`kvminit` 与 `kvminithart` 的名字相近，恰好分别对应这两个角色。

6.6 第一个进程为什么特殊

普通进程可以从已有进程创建，但系统最初没有可用父进程。经典 userinit 人工建立第一个进程：分配管理结构，准备用户页表和寄存器现场，装入一小段 initcode，让它成为可运行状态。

调度器第一次选择它时，从预设的 forkret 进入首次内核执行路径。必要准备完成后返回 U 模式，initcode 发出 exec 请求，装载真正的 init 程序。init 连接控制台并启动 shell，最终出现用户提示符。

某些版本省去 initcode，改为在首次内核路径直接装载 init。阅读时先检查自己版本中哪条链实际存在。两者共同解决的问题都是：从“还没有用户进程”开始，构造一个能够正确进入用户态的执行环境。

6.7 停下来推演

问题 1：为什么先要栈，再进入 C？ C 函数按调用约定使用栈保存部分状态；未准备 sp 就调用，可能写到不属于自己的位置。

问题 2：为什么其他 hart 不能只等一个数值变成 1 就忽略内存顺序？ 它们还需要确认共享对象的初始化内容已经可见。数值通知与内容发布必须按协议衔接。

实验。 搜索启动提示、其他 hart 启动提示和 init 启动 shell 的提示，分别判断属于内核还是用户程序。若加入阶段日志，应在打印设施可用之后，并尽量只记录少数关键点。

源码路线：entry.S→start.c→main.c→userinit→forkret。先解释每次交接建立了什么条件，再核对具体寄存器。

第三篇 / 内存与执行现场

07 虚拟内存
同一个地址为什么互不干扰

学习目标：从简单映射理解页和页表；完成一次地址翻译；理解权限、页表切换及特殊入口映射。

7.1 先用一张小表解决冲突

假设程序 A 和 B 都想把数组放在地址 100。若两者直接使用同一物理位置，数据会冲突。一个解决办法是给每个程序自己的编号表：A 的 100 对应物理位置 10000，B 的 100 对应 20000。

编号表的想法就是虚拟内存的入口。程序使用虚拟地址，CPU 按当前映射得到物理地址。这个映射属于执行环境的一部分，切换进程时应使用正确的映射。

同一个虚拟地址未必对应同一物理位置；不同虚拟地址也可能按明确设计指向同一物理页。后一种共享需要严格管理所有权和权限，不能仅凭数值判断对象是否共享。

7.2 为什么按页映射

若为每个字节单独保存转换关系，管理信息太大。操作系统和硬件通常按固定大小的页管理地址范围。xv6 常用一页 4096 字节，也就是 4 KiB。

某个虚拟地址可以拆成“它属于第几页”和“它在这一页内的第几个字节”。前者用于查映射，后者叫页内偏移。映射决定物理页基址，再加上相同偏移就得到物理地址。

例如十进制地址 5000：$5000 = 1 \times 4096 + 904$。它属于从 0 开始编号的第 1 个虚拟页，在该页偏移 904。若这页映射到物理基址 40960，最终位置就是 41864。40960 是 4096 的整数倍，满足这里的页对齐要求。

术语卡：页基址是一页开始的地址；页内偏移是从该起点向后多少字节。4096 字节页的基址是 4096 的整数倍。



图 7-1 一页映射决定起点，页内偏移保持不变

7.3 页表项既保存位置，也保存权限

页表是保存映射的内存数据结构，其中一条记录叫页表项，英文 page table entry，简称 PTE。它给出目标页相关信息，并用若干位表示有效性和访问权限。

标志	初学时怎样理解	缺少时可能发生什么
V	这条记录有效	不能按普通有效映射使用
R	允许读取	不满足规则的读取触发故障
W	允许写入	写入受限，不能随便改内容
X	允许执行指令	不能把该页当代码执行
U	用户态可访问	普通用户不能直接访问

权限让程序的代码页可以读和执行、但通常不能写；数据页可以读写、但通常不执行；内核管理数据不授予普通用户访问。具体判断还要结合模式和体系结构规则，不能认为 S 模式可以忽略所有页权限。

访问一个没有合适映射或权限的地址会引发页面错误，简称缺页或 page fault，具体也可能是权限故障。它通知内核“这次地址访问不能按当前规则完成”，不保证一定是物理内存耗尽。

7.4 为什么是一棵三级表

RISC-V Sv39 是一种虚拟内存方案。xv6 的基础映射使用三级页表，每张表有 512 个条目，每个条目占 8 字节，恰好 512 × 8 = 4096 字节。

可以把它想成“总目录、分目录、具体映射”。根页表先存在，只有某个地址范围真的需要映射，才分配相应下级页表。大量未使用范围不用配齐所有表项对应的下级表，因此可以节省内存。

三个索引各需要 9 位，因为 2 的 9 次方是 512；页内偏移需要 12 位，因为 2 的 12 次方是 4096。合起来是 39 位。这里讨论数学上的乘方，C 表达式中的 ^ 是按位异或，并非乘方。地址高位还有规范要求，本书先使用 xv6 采用的低地址范围，不把任意 64 位数都视为合法地址。



图 7-2 Sv39：拆分虚拟地址，再逐级寻找叶子项

7.5 一步步翻译 0x12345

把虚拟地址 0x12345 向下取整到页边界，得到 0x12000，偏移是 0x345。三个索引按位分解后，最高层 L2 为 0，中间 L1 为 0，最低 L0 为 0x12，也就是十进制 18。

第一步在根表找第 0 项，取得下一张表；第二步在下一张表找第 0 项；第三步在最低级表找第 18 项。假设这个叶子项给出的物理页基址是 `0x81000000`，最后加偏移，结果为 `0x81000345`。

内容	本例结果	它的作用
L2 索引	0	在根页表选择下级表
L1 索引	0	在中间表继续选择
L0 索引	0x12	找到最后一级映射
页内偏移	0x345	在物理页中确定字节位置

内核的软件函数 walk 也会沿这样的结构查找或创建表项。正常用户访存时，则由硬件翻译，TLB 命中时还可省去完整遍历。TLB 是保存近期地址翻译结果的缓存，不是另一份用户数据。

7.6 当前页表由 satp 指向

CPU 需要知道当前从哪张根表开始翻译。RISC-V 的 satp 控制寄存器包含页表根等信息。安装另一张页表，就可能让相同虚拟编号通向另一组物理页。

TLB 可能仍保留旧结果，所以修改翻译规则时要按要求执行 `sfence.vma` 等同步操作。可以把它理解成：地图改了，还要让处理器停止使用不再适用的旧路线。具体需要刷新哪些执行者的结果，要结合实现与并发情况分析。

先记住职责分工：PTE 保存关系，satp 选择根，TLB 加速已有翻译，sfence.vma 参与更新后的正确衔接。它们不是四个互相替代的东西。

7.7 切换页表时，入口代码为什么不能消失

用户进程运行时使用自己的页表，内核主体使用内核页表。发生陷阱后，硬件刚进入 S 模式时还没有自动换页表，入口代码必须在当前映射下也能执行。

xv6 把一小段入口与返回代码称为 trampoline，可以理解为跨越两套映射的桥。它在用户和内核页表的同一虚拟地址映射到同一物理代码页，使改写 satp 前后都能继续取指。

用户页表还映射一个保存用户寄存器现场的 trapframe。trampoline 与 trapframe 都没有 U 权限，用户不能随意访问；提升到 S 模式后的入口代码按规则使用它们。trapframe 先记作“用户现场保存区”，第 9、10 章再展开字段。

7.8 页表本身也要分配和释放

一棵表是内存对象，不是 CPU 凭空提供的无限结构。创建新地址空间需要根表，映射稀疏范围时还可能增加中间表；映射普通用户数据又需要另一些物理页。

因此释放要分清两类资源：被映射的数据页，与装着 PTE 的页表页。删除映射未必等于释放数据页，尤其共享代码页不能被每个进程都当作独占对象释放。叶子映射清理之后，再递归释放表结构。

进阶提醒：体系结构支持更大粒度的页，较高级条目也可能是叶子。xv6 基础路径主要按最末级 4 KiB 页解释；不要把这个教学约定误认为 Sv39 的全部能力。

7.9 停下来推演

问题 1：两个进程都用 0x1000，为什么不一定冲突？ 地址翻译从各自的根表开始，最终物理页可以不同。数值相同不代表映射相同。

问题 2：页表项有效就一定能写吗？ 不一定，还要有与当前访问相匹配的权限。只检查 V 不足以决定用户写入合法。

纸笔练习。 用 4096 字节页计算地址 8195 的页号与偏移。参考：8195 = 2 × 4096 + 3，页号为 2，偏移为 3。再假设映射到物理基址 32768，最终地址为 32771。

源码路线：先看 memlayout.h 中的布局，再看 walk、mappages、proc_pagetable。第一次阅读只跟一个地址，不同时追整棵树的所有分支。

第三篇 / 内存与执行现场

08 内存从哪里来，又怎样归还

学习目标：区分物理分配与虚拟映射；逐页计算增长和收缩；理解清零、用户复制、懒分配与所有权。

8.1 申请空间其实包含两个问题

程序希望多使用一页，内核至少要解决：哪一页真实 RAM 归它使用；程序使用哪个虚拟地址找到这页。前者由物理页分配器处理，后者由页表映射处理。

只取到物理页，却没有用户映射，程序不能通过预期地址使用它。只写出一个看似有效的地址，却没有正确物理归属，也不会产生可安全使用的存储。分配与映射需要合作，但不能混成一个概念。

8.2 用空闲链表管理物理页

启动时，内核把可分配区域切成一页一页。空闲页暂时不保存任何使用者的有效对象，因此可以借用它开头的几个字节，保存“下一张空闲页在哪里”。多个空闲页串起来就是链表。

kalloc 从链头取出一页，把所有权交给调用者；kfree 把符合规则的页放回。多核可能同时操作链表，必须同步，否则两位调用者可能取到同一页，或者链表节点丢失。

术语卡：所有权回答“现在谁有责任使用和最终释放资源”。它不只是一个 C 类型，还包括对象何时有效、是否可共享以及释放条件。

释放后的指针即使数值不变，也不再代表你拥有那页。别人可能把它用作页表、用户数据或内核对象。继续使用旧指针叫释放后使用，英文 use-after-free，会破坏不相关的对象。

8.3 为什么分配后还要清零

一页 RAM 之前可能装着另一个进程的内容。若原样交给新用户，新用户可能读到他人的数据。因此在暴露给用户前，要确保新内存按约定初始化，通常需要清零。

经典 kalloc 为调试可能填入特定字节，并不承诺返回全零页。uvmalloc 等负责创建用户页的调用者再清零。释放时填特殊值，也主要是帮助暴露错误，不能替代对新用户的初始化要求。

例如申请一页作为页表时，未使用条目应处于无效状态。若不清零，残留字节可能碰巧像一个有效 PTE，硬件或内核便可能沿着垃圾地址继续查找。这说明初始化也取决于对象将被解释成什么。

8.4 sbrk 与按页舍入

xv6 的用户分配器可以通过 sbrk 请求扩大内存范围。基础接口成功时返回旧边界，失败返回 `(char *)-1`。负参数尝试缩小，不能把“负数”本身直接当作错误。

进程的 sz 字段以字节记录一个用户地址空间上界，不等于单独的堆大小。物理页按 4096 字节分配，边界只多出 1 字节也可能需要下一页。

逻辑范围变化	要覆盖哪些普通页	结果
0→4096	第 0 页	正好一页
4096→4097	第 0、1 页	增加第二页
4097→4096	第 0 页	第二页可回收
8192→4097	仍需第 0、1 页	没有减少整页

这个表是从空范围开始的纸笔例子。真实用户程序已有代码、数据和栈，运行实验应以 sbrk(0) 返回的当前边界为起点。不要真的假定普通程序一开始的大小为零。

8.5 内核为什么不能直接使用用户指针

用户把一个数值当作缓冲地址传进来，可能是有效数组，也可能是未映射区域。内核运行时还可能使用另一张页表，用户虚拟编号不能简单强制转换成可访问的内核指针。

copyin 表示把用户数据复制到内核，copyout 表示把内核数据复制到用户，copyinstr 用于复制有最大长度限制、以零结尾的字符串。名字里的 in/out 以“进入或离开内核”为参照。

这些函数按用户页表翻译地址并验证条件，然后通过内核能够使用的物理内存映射完成复制。需要什么权限应核对实际实现，向用户写数据尤其要检查可写性。没有终止符的字符串也必须受到长度上限约束。

8.6 亲手算一次跨页复制

假设源地址为十进制 4093，长度 10 字节。第一页范围 0 到 4095，起点后只剩 3 字节；剩余 7 字节落入第二页。一次把 10 字节全当成同一页处理，会绕过对第二页的检查。

轮次	当前用户地址	本轮最多处理	之后还剩
1	4093	3 字节，抵达第一页末尾	7 字节
2	4096	重新查第二页，再处理 7 字节	0 字节

若第二页不存在，复制可能在第一轮已经完成 3 字节后失败。错误返回不自动代表没有任何修改。设计系统调用时，需要明确是否允许部分副作用，不能凭返回 -1 就推断它像数据库事务一样全部撤销。

计算地址与长度时也要防止溢出：很大的地址加上长度可能绕回较小数值。检查整个范围不能只写一个不考虑整数表示的简单比较。

8.7 进阶：先登记，等使用时再分配

基础实现通常在 sbrk 扩大范围时立即分配所需页面。懒分配则先记录允许使用的范围，等程序第一次访问某个尚未存在的页时，引发页面错误，再分配这一页并建立映射。



图 8-1 相同申请，不同的物理页分配时机

如果程序申请 1 MiB 却只触碰 4 KiB，懒分配有机会减少实际分配。但它把工作推迟到首次访问时，也增加了错误处理要求：缺页时内存可能不足，fork 和退出要能处理未分配空洞，用户复制也要认识合法的懒页。

缺页地址小于 sz 还不够证明合法。保护页、对只读代码的写入或特殊映射都不能被随便改成新可写页。处理器报告“访问不能完成”，内核仍要判断原因。

本书将这部分作为扩展；没有懒分配的基础版本，sbrk 后不访问也已经取得了新数据页，实验结果自然不同。

8.8 失败以后，谁清理已经取得的页

假设本次准备增加三页，前两页成功，第三页分配失败。调用者不能留下一个声称增加三页、实际只有两页的新范围，也不能把已经取得的两页永久丢掉。

应追踪资源清单：每页是否分配，是否已映射，由谁解除映射并释放。用户可见大小要恢复到约定状态。某些实现不会立即剪除所有已经建立的空中间页表，但它们仍有所有者，最终销毁页表时应被回收。

资源回收的正确性比“成功路径能运行”多一步：失败发生在任意中间位置，都要知道每个对象现在归谁。

8.9 停下来推演

问题 1：为什么 kalloc 后常还能看到 memset？ 分配只交接一页，未必清零。调用者必须按新对象用途初始化，尤其避免把旧数据交给用户。

问题 2：把范围从 5000 缩小到 100，回收几页？ 在这个从零计数的简化例子中，原范围覆盖两页，新范围只需第一页，因此可回收第二个数据页。

实验。 先在纸上给一页写“空闲→归分配器调用者→被映射→解除映射→回收”，每一步标出责任人。随后只阅读 kalloc、uvmalloc 和 uvmunmap 中对应的动作，检查失败分支有没有交接遗漏。

源码路线：kalloc.c→growproc→uvmalloc/uvmdealloc→copyin/copyout。懒分配相关代码放在基础路径看懂之后。

第三篇 / 内存与执行现场

09 进程怎样暂停、切换和退出

学习目标：建立完整的进程对象图；区分两种保存现场；解释 fork 返回、调度状态与 wait 回收。

9.1 从两个计数程序理解运行现场

程序 A 正把 i 从 3 增加到 4，程序 B 正等待文件数据。它们不仅有不同 PID，还各自拥有代码看到的内存、当前执行位置、寄存器、调用栈以及资源引用。内核把这些信息组织到进程结构及相关对象中。

可以把暂停理解为把某条执行路线暂时放在一边。用户内存不必整份搬走，但继续执行所需的位置和寄存器必须能找回来。此时 CPU 可以恢复另一条路线。

9.2 为什么一个进程需要用户栈和内核栈

用户栈保存用户函数调用使用的状态。进入内核后，系统调用自己的局部变量和调用链应放到受保护的内核栈，不能继续信任用户随意提供的栈地址。

用户程序执行 read 时，先进入内核；内核可能沿文件系统走到磁盘等待处。即使进程暂时让出 CPU，这条内核调用链也必须保留，稍后才能从等待结束的位置继续。每进程内核栈正是为此提供空间。

对象	里面是什么	什么时候用
用户栈	用户函数调用的状态	普通用户代码执行
内核栈	当前系统调用等内核调用链	处理服务、等待后继续
trapframe	用户寄存器和返回信息	用户态与内核态往返
context	内核切换所需的一组寄存器	进程内核线程与调度器切换

9.3 trapframe 与 context 保存的是两层现场

假设用户计算中调用 read。进入内核时，用户寄存器要保存到 trapframe，稍后返回用户才能接着运行。如果 read 又因为磁盘未完成而睡眠，内核当前执行位置与栈指针还要保存到 context，让内核代码稍后继续。

因此可以出现“用户现场已经在 trapframe，内核现场又停在 context”的情况。它们不是相同信息的两个名字，而是处于两层不同边界。



图 9-1 用户返回现场与内核等待现场保存在不同位置

普通函数调用约定解释了 context 为什么只保存 ra、sp 与被调用者保存寄存器等项目。用户陷阱发生在任意指令时，用户没有机会先按普通调用准备，trapframe 因此需要保存更完整的用户通用寄存器现场。

9.4 用状态图认识调度

调度器不能选择已经退出或仍在等待事件的进程。内核用状态表达它是否具备继续运行的条件。

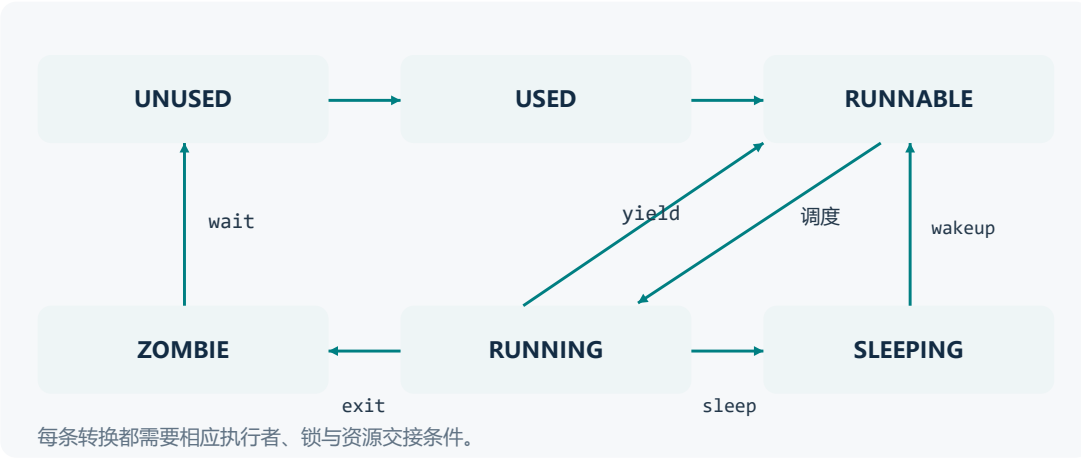


图 9-2 进程状态与执行责任

UNUSED 表示槽位空闲；USED 表示正在构造；RUNNABLE 表示准备好但尚未获得 CPU；RUNNING 表示已被某个 CPU 运行；SLEEPING 表示等待条件；ZOMBIE 表示已经结束但退出信息尚未由父进程回收。

“可运行”与“正在运行”尤其要分清。一个进程读空管道时会睡眠，数据到来后变为可运行，之后还要等调度器选择，才真正继续执行。唤醒不等于立刻获得 CPU。

9.5 一次切换究竟保存什么

进程调用 yield 表示愿意让出 CPU。经典路径在规定锁条件下把状态改为 RUNNABLE，再通过 sched 与 swtch 保存进程内核 context，恢复调度器 context。调度器稍后选一个进程，反向恢复其 context。

栈指针一换，后续函数使用的是另一条已经存在的栈。切换不复制全部用户内存，也不重新执行 main。第一次被运行的进程则由构造好的 context 指向 forkret，以便从特定起点开始。

实际切换涉及 p->lock 的专门交接协议，避免两个 CPU 同时使用同一进程现场。第 12 章再看锁的证明。现在只要知道“状态和现场交接必须作为一个协调过程”，不能先随意标成可运行再慢慢保存。

9.6 fork 为什么让一份代码走出两条路线

fork 从当前进程创建子进程。基础 xv6 复制用户内存和用户寄存器现场，为子进程准备独立管理结构，并为共享文件等增加引用。成功后父进程获得子 PID，子进程获得 0。

xv6 用户程序 / 按注释区分完整程序与片段

```
// xv6 用户程序的逻辑片段。
int pid = fork();
if (pid < 0) {
  printf("fork failed\n");
} else if (pid == 0) {
  printf("child\n");
  exit(0);
} else {
  wait(0);
  printf("parent\n");
}
```

子进程从 fork 后的返回点继续，不是从 main 重头运行。内核通过子 trapframe 的 a0=0 准备子返回值，父调用的返回值则设为子 PID。两者恢复后使用同一段分支逻辑，却进入不同分支。

本例父进程 wait 等子进程结束后才打印 parent，因此形成特定先后关系；若去掉 wait，父子输出先后一般不固定。不要把一次运行的调度顺序当作必然规律。

9.7 复制内存与共享资源同时存在

父子各自修改普通变量，基础复制内存机制下通常互不影响；但文件表中的指针可以指向同一个 file 对象，文件偏移可能共享。fork 不是“所有东西全复制”或“所有东西全共享”中的任一个极端。

经典 xv6 的进程内核栈页在启动时集中准备并长期保留，allocproc 使用相应位置。不要据“创建进程要有栈”推断每次 fork 一定重新分配栈物理页，或每次 wait 一定释放它。对象需要存在与何时实际分配，是两个问题。

9.8 为什么 exit 后还要 wait

子进程结束时，父进程可能还需要知道它的编号与退出状态。exit 关闭文件等资源，处理子进程归属，记录退出信息，并停止执行。它进入 ZOMBIE，保留尚待回收的部分状态。

wait 由父进程查找已结束的子进程，取走退出状态，释放剩余用户内存、页表和管理资源，使槽位重新空闲。若有子进程但尚未退出，父进程等待后重新扫描；没有子进程则按接口返回失败。

父进程先退出后，仍存活的子进程成为孤儿，由 init 接管回收责任。孤儿可能还在正常运行；僵尸已经执行结束。它们描述不同情形。

9.9 停下来推演

问题 1：进程睡眠是否意味着 CPU 也停了？ 不一定。等待者让出 CPU，调度器可以运行其他 RUNNABLE 进程。

问题 2：yield、sleep、exit 的区别？ yield 后仍具备运行条件；sleep 等条件变化；exit 结束执行，等待回收。三者都可能让出 CPU，但下一步资格不同。

纸笔练习。 画 A 运行、B 可运行、C 睡眠的场景。A 让出后可能选择 B；B 等待管道后变睡眠；A 仍可再次被选中。把状态变化和选择动作分开画，就能解释“没有固定顺序，但不是任意顺序”。

源码路线：proc.h→allocproc→fork→yield/sched→scheduler→exit/wait。每次只给一个对象填写状态和资源清单。

第三篇 / 内存与执行现场

10 陷阱
CPU 怎样把控制权交给内核

学习目标：理解 trap 的硬件入口；分清异常、中断和系统调用；沿用户现场保存到恢复走一遍。

10.1 为什么需要特殊入口

如果用户只需把 PC 改成某个内核地址就能取得权限，隔离便不可靠。硬件需要提供受控机制：发生特定事件时，记录原因与返回位置，按规则进入预先设置的内核入口。

这类控制转移统称 trap，中文常译陷阱。词里的“陷”不表示一定出错：正常系统调用、正常时钟中断都可以走这套机制。具体是否错误，要看原因与当前进程状态。

10.2 三个现象，两种原因类别

异常通常由当前指令触发，例如访存没有合适映射。中断是时钟、设备等带来的异步事件。用户执行 ecall 请求服务，会产生一种同步异常，供操作系统实现系统调用。

场景	与当前指令的关系	通常要做什么
调用 getpid	主动执行 ecall	分派服务并返回结果
设备完成磁盘读	完成时异步通知	处理结果并唤醒等待者
写入不允许的地址	当前访存不能完成	修复允许的情况或结束进程

从 U 模式执行 ecall 时，RISC-V 的原因是来自 U 模式的环境调用，常见同步原因码为 8。它进入 S 模式处理，并不因此变成“来自 S 模式的 ecall”。来源与处理位置要分开。

10.3 硬件先记录哪些信息

在进入 S 模式陷阱时，sepc 记录相关执行位置，scause 记录原因，stval 可能提供故障地址，sstatus 保存来源权限和中断相关状态，stvec 指向处理入口。它们是控制状态，并非普通用户变量。

硬件不会自动把所有通用寄存器保存到一块 C 结构体，也不会自动切换 satp 和 sp。若入口代码立即把某个用户寄存器拿来当临时变量，就可能丢失用户原值，因此最前面一小段汇编必须格外小心。

> 术语卡：保存现场是把稍后恢复所需的状态放到可靠位置；恢复现场是把这些值重新装回。保存不是重新计算用户做过的工作，而是保留它停下时的状态。

10.4 uservec 把不适合 C 的入口变成可用环境

用户运行时，stvec 指向 trampoline 中的 uservec。陷阱发生后，CPU 已有 S 模式权限，但仍使用用户页表。trampoline 的共同映射保证入口取指可继续，受保护的 trapframe 保存用户寄存器。

uservec 再从事先填写的 trapframe 字段取得内核栈、内核页表与 C 处理函数地址，切到内核环境并调用 usertrap。这样，普通 C 函数才可以依靠正确的栈与映射工作。

例如用户 a0 原来保存参数，但入口也需要寄存器来定位 trapframe，就要先利用 sscratch 等约定暂存用户值。具体交换方式随版本不同，先抓住“不能弄丢用户原值”的目的即可。

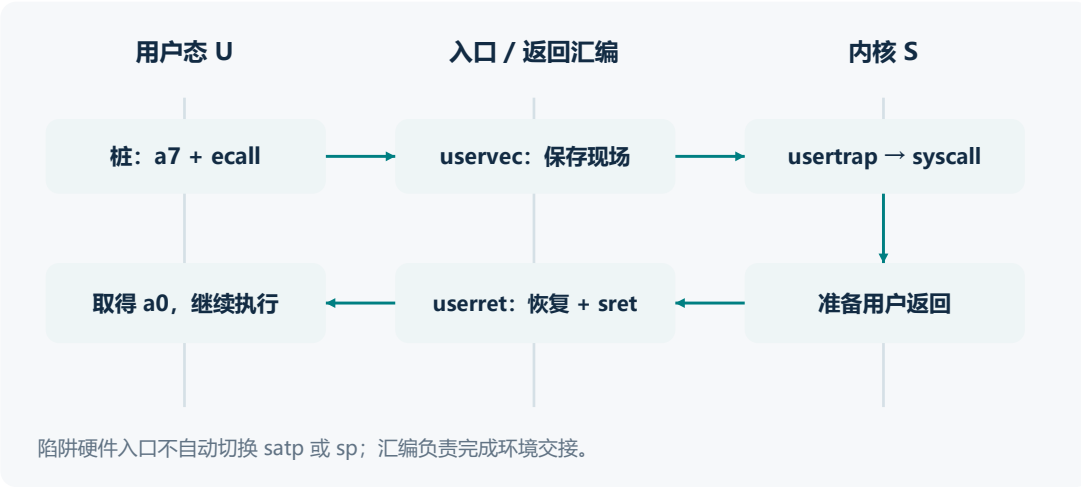


图 10-1 用户态、入口汇编与内核处理的往返

10.5 让最简单的 getpid 走完整条路

第一步，用户桩把 getpid 的系统调用编号写入 a7，执行 ecall。第二步，硬件记录原因和位置，跳到 uservec。第三步，汇编保存用户寄存器，切到内核栈和页表。第四步，usertrap 识别系统调用，再调用统一分派函数 syscall。

第五步，syscall 根据编号调用 sys_getpid，取得当前进程 PID。第六步，结果写回 trapframe 的 a0。第七步，返回准备函数设置回到 U 模式所需控制状态。第八步，userret 切回用户页表、恢复寄存器、执行 sret，用户程序读到返回值。

这八步先用中文复述，再附函数名。getpid 本身只取一个编号，适合观察公共入口；read 等服务还会在相同入口后增加文件、内存复制和等待设备的工作。

10.6 为什么 ecall 要前移，缺页却要重试

ecall 的请求已经处理，返回时应执行它后面的指令。保存的返回位置原本指向 ecall 本身，xv6 将进程保存的 epc 加 4，因为 ecall 是 4 字节指令。若不前移，就会再次发出同一请求。

可修复缺页则不同：原来的 load/store 尚未完成。内核补好合法映射之后，应保留原 PC，让那条访存重新执行。这时若错误地加 4，就跳过了程序原本需要的工作。

事件	原指令是否需要再次执行	返回位置的处理
已处理的用户 ecall	不需要	跳到 ecall 后
可修复的页面错误	需要	保留原访存位置
普通中断	按被打断位置继续	不按 ecall 的规则额外前移
不可修复用户错误	不再恢复该错误执行	结束当前进程

代码中可能先修改 trapframe 的 epc，稍后才写硬件 sepc。讲解里的“前移 sepc”是一种简写，阅读时要辨认真正被改的是哪个保存位置。

10.7 为什么内核态还需要另一套入口

内核执行过程中也可能收到设备中断。此时已在内核页表与内核栈上，不需要像用户入口那样从不受信任用户环境开始准备。xv6 使用 kernelvec 和 kerneltrap 处理这类入口。

若中断导致调度，返回时还要恢复被打断内核代码所需的执行状态。内核态异常往往意味着内核自身不变量被破坏，处理通常比普通用户错误更严格。不能把用户传错指针与内核自己写坏内存视为同一类责任。

10.8 停下来推演

问题 1：trap 是否一定是程序错误？ 不是。它是控制权转移机制，系统调用和正常设备中断都是日常使用。

问题 2：为什么刚进入 S 模式还不能立刻调用任意内核 C 函数？ 正确内核栈和页表尚未自动就位，入口汇编要先建立这些条件。

练习。 给 getpid、时钟中断、合法懒页访问各画一条四格路线：“发生什么→保存什么→处理什么→回到哪里”。重点检查哪一条需要前移返回位置，哪一条需要原地重试。

源码路线：uservec→usertrap→syscall→用户返回准备→userret。内核态入口放在用户路径看懂之后比较。

第四篇 / 接口、并发与通信

11 系统调用 把用户请求接到内核服务

学习目标：追踪参数与返回值；区分提取和验证；理解系统调用各连接点及失败清理。

11.1 再看 read 的三个参数

Unix 风格接口常写作 `read(fd, buf, n)`。fd 是已经打开的输入对象编号，buf 是程序准备接收内容的地址，n 是最多希望读取的字节数。返回值通常是实际读到的字节数。对正长度的普通文件或管道读取，0 通常表示输入结束，负值表示错误；请求 0 字节时返回 0，则不能据此判断输入已经结束。

这里的三个参数含义不同：fd 不能超出当前进程的可用槽位；buf 不是内核可以任意直接访问的地址；n 不能让范围计算失控。即使参数类型都像简单整数，检查责任仍然不同。

一次 read 可能少于 n 字节，用户程序应循环处理直到完成自己的目标或收到结束条件。系统调用提供实际完成的结果，不能只凭函数名假定“一次就填满数组”。

11.2 用户桩为什么只有几条指令

用户代码调用 read，首先进入用户程序中链接好的一个短入口，常称系统调用桩。它把对应编号放进 a7，再执行 ecall。参数已经按约定放入 a0、a1、a2 等位置，桩无需重新实现文件读取。

进入内核后，用户寄存器被保存到 trapframe。内核分派函数从 a7 找到编号，处理函数再取出参数。返回时结果放入 a0，使用户代码看起来像获得了普通函数返回值。

可以把路径分成“传请求”和“做业务”两部分。用户桩只处理跨边界协议，sys_read 与文件层解释具体读取对象；页表复制函数处理用户内存；驱动再处理设备。这些分层使每层的职责都能单独检查。

11.3 五个位置如何连成一个接口

位置	初学时先看什么	它解决什么问题
user/user.h	用户函数声明	C 调用的名称和类型
user/usys.pl	是否登记该调用	自动生成用户汇编桩
kernel/syscall.h	编号是否唯一	用户和内核用同一服务编号
kernel/syscall.c	编号对应哪个函数	验证并分派请求
sysproc.c、sysfile.c 等	参数、实现与返回	完成实际服务

编号、声明、桩和实现需要一致。漏用户桩可能出现链接缺符号，漏分派可能得到未知系统调用，编号冲突可能进入错误服务。仅写一个叫 sys_xxx 的 C 函数，不会自动形成用户可调用接口。

第 4 章的函数指针表在这里派上用场。表项类型相同，处理函数可以统一声明为返回 uint64、无显式形参；因为真实参数从当前进程保存的用户寄存器中提取。

11.4 argaddr 为什么不能替你证明安全

argint 取整数，argaddr 取地址数值，argstr 取得有长度约束的字符串。前两者主要回答“用户传了什么值”，不等于“这个值在当前请求里合法”。

用户把一个很大的数当作地址，argaddr 仍可成功取到它。等 copyout 要把内容写进用户内存时，才需要验证页表、范围和权限。fd 的验证则在找到 file 的层进行，是否可读又由 file 等对象状态决定。

argstr 将路径复制到有限的内核缓冲区，使查找使用已取得的字符串，而不把任意用户地址当作稳定内核数组。复制还必须找到结尾或在最大长度处失败，避免无限向后扫描。

11.5 一次非法缓冲区请求会停在哪一层

假设用户传入一个从未映射的地址，请求 read 10 字节。系统调用入口仍然可能正常，fd 也可能正确。真正向用户目标复制时，复制层发现对应页面无效，于是按实现返回失败。

这说明“进入了内核”不代表请求已经被批准。检查可以分布在多层，每层验证自己负责的条件。对普通用户错误，合理目标是拒绝请求或结束当前进程，使其他进程和内核继续工作。

部分失败还要看副作用。若先处理了几字节、随后遇到无效页，缓冲区可能部分改变，某些底层对象也可能推进。不要把所有错误返回都解释成“此前步骤完全没有发生”。

11.6 练习接口：为 getpid 增加一个别名

在练习分支中，可以增加 `mypid()`，行为就是返回当前 PID。先写用户声明 `int mypid(void)`，选择尚未使用的系统调用编号，登记用户桩，在内核分派中声明并连接 sys_mypid，处理函数返回当前进程 PID。

这个例子没有用户指针，因此适合专心检查整条接线。它不是需要取代 getpid 的新功能，也不必为了“测试更多”凭空增加缓冲区参数。

用户测试应比较同一进程的 mypid 与 getpid 是否相等。fork 后父子各自比较仍应相等，而父子进程编号不同。测试不要写死“第一个测试进程 PID 必须是 3”，因为执行顺序可能变化。

加入用户命令时还要登记到 UPROGS，使构建系统生成并打包它。编辑一份未参与构建的代码，不会改变实际运行行为。

11.7 新接口要先规定失败语义

如果将来新增输出结构体接口，应先规定：用户最多提供多少字节，长度为零怎么办，容量不足返回什么，非法地址时是否允许部分写入。向用户复制的结构体应只包含公开字段，并初始化填充字节。

不能把整个内核进程结构直接复制出去，里面包含内核指针、锁与内部布局。接口设计的目的，是向用户提供可理解、可检查的结果，而不是泄漏实现中的全部内存。

资源也需要失败清单。例如 open 可能先取得 inode，再分配 file，最后找 fd 槽位。若最后一步失败，前面取得的引用和对象必须归还。每个 return -1 前都值得追问“现在还有谁被我持有”。

11.8 停下来推演

问题 1：系统调用函数没有 C 形参，参数去哪了？ 已在 trapframe 的用户参数寄存器字段中。处理函数按约定提取。

问题 2：用户库提前检查过地址，内核可以不查吗？ 不可以。用户可以修改自己的程序，绕过库或直接构造请求；内核边界必须独立验证。

练习。 从 read 画六个方框：用户调用、桩、陷阱入口、分派、文件读取、用户复制。在每个框下写它理解哪些信息。参考：桩理解编号协议，文件层理解对象类型，复制层理解用户地址与权限。

源码路线：user.h→usys.pl→syscall.h→syscall.c→sys_read。最后再加 fileread 与 copyout，把接口接到子系统。

第四篇 / 接口、并发与通信

12 并发与锁
一起使用资源为什么会出错

学习目标：拆开非原子步骤；从不变量理解锁；推演条件等待、丢失唤醒与死锁。

12.1 一行加法为什么可能丢失更新

假设两个 CPU 同时执行共享计数器 `count = count + 1`，初值为 5。每个都可能分成读旧值、加一、写回。下面的顺序完全可能发生：

时间	CPU A	CPU B	共享 count
1	读到 5	尚未读取	5
2	等待后续执行	也读到 5	5
3	算出 6	算出 6	5
4	写回 6	尚未写回	6
5	已完成	写回 6	6

本来做了两次递增，结果却只增加一次。原因是两次操作交错时都基于同一个旧值。依赖执行先后才能保持正确、但又没有恰当协调的情形，常表现为竞态问题。

12.2 先写不变量，再圈出临界区

不变量是正确状态必须保持的关系。例如“同一空闲物理页不能同时交给两位使用者”“每次 PID 分配取得唯一编号”。临界区是需要作为受保护过程访问共享状态的代码区域。

互斥意味着同一时刻只有获得资格的执行者进入相应临界区。锁保存这种资格：一个执行者取得锁、完成共享修改并释放，其他执行者按相同协议等待。

如果有人修改相同字段却不遵守锁协议，锁就无法保护它。把一段代码包进 acquire/release，并不会自动修复其他访问路径，也不会自动决定锁该保护哪些关系。

12.3 原子操作、自旋与内存可见性

取得锁这件事本身必须可靠，不能也分成“先看看没人、再写我占用”而让两人同时通过。原子操作提供不可被另一个同类操作拆开的状态转换，供锁实现竞争所有权。

自旋锁的等待者会循环尝试，暂时占用 CPU。适合非常短的临界区，例如修改链表头。若等待可能很久，CPU 就会白白耗在检查上；某些锁关系还可能导致真正能释放资源的路径无法运行。

锁还涉及可见性：一位执行者完成修改后，另一位按协议获得锁，应能看到相关结果。编译器和 CPU 可以在允许范围内调整访问顺序，所以锁实现还要使用适当的原子语义和屏障。只给变量加 volatile 并不能替代这套保证。

12.4 为什么自旋锁常要配合关闭本核中断

假设某 CPU 持有锁 L，突然收到中断；中断处理也想取得 L。它会等原代码释放，可原代码正被它打断，根本没有机会继续释放，于是自己等待自己。

xv6 通过 push_off/pop_off 管理这类临界区的本 CPU 中断状态，并支持嵌套。关闭本核中断并不能阻止其他核访问共享数据，因此还需要真正的互斥锁。

不要由此得出“中断越少越安全”。长时间关闭中断会影响响应和进展；应只在正确范围内使用规定协议。

12.5 睡眠锁与等待条件分别是什么

睡眠锁是一种互斥工具：锁被占用时，等待进程可以让出 CPU，适合 inode、磁盘缓存内容等可能涉及较长等待的资源。

等待条件则是“我需要某件事变为真才能继续”，例如管道有数据、磁盘请求完成。条件由共享状态表示，条件锁保护检查和修改，channel 用来标识等待哪件事，wakeup 通知相应等待者。

channel 可以是一个约定好的地址值，它本身不一定装着一条消息。通知也不等于数据已经归某人所有：醒来后仍要重新取得条件锁，再检查共享状态。

12.6 丢失唤醒是怎样发生的

错误读者先检查“没有数据”，然后释放条件锁；写者写入数据并通知；读者却在通知之后才登记自己睡眠。此时数据可能已经存在，读者仍睡下，而不再有下一次通知。

这就叫丢失唤醒。问题在于“检查条件”和“开始等待”之间存在一个无法正确衔接的窗口。正确协议必须让通知者不可能从这个窗口中穿过而完全错过等待者。

伪代码 / 只用于机制推演

```
取得条件锁
while 条件不成立:
    sleep(channel, 条件锁)
    // sleep 返回时，重新持有条件锁
使用已经再次检查过的资源
释放条件锁
```

经典 sleep 在仍保护条件时取得当前进程锁，再释放条件锁、登记睡眠并切出。wakeup 检查相应进程也要使用进程锁，两者通过锁交接衔接。第 9 章的进程状态于是和条件锁联系起来。

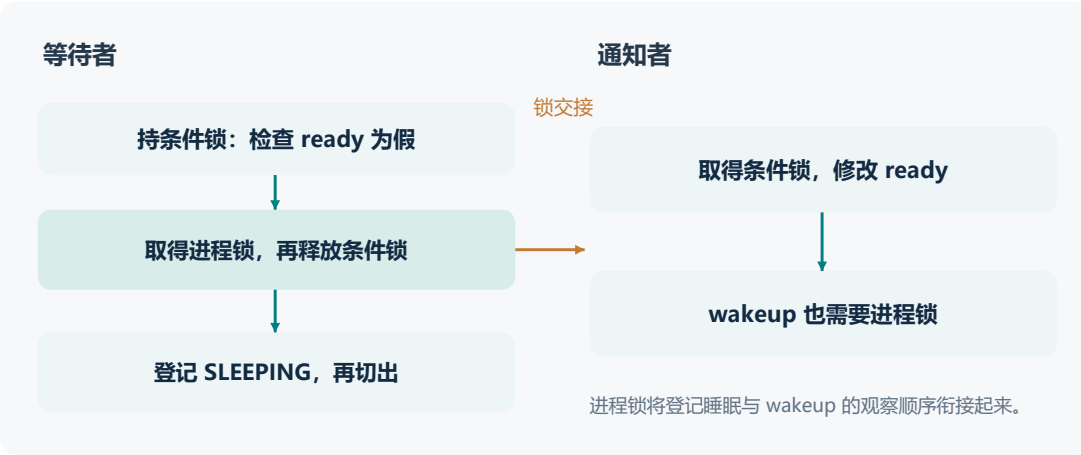


图 12-1 条件锁与进程锁衔接，封住丢失唤醒窗口

12.7 为什么醒来后使用 while

通知通常只表示“条件可能变了”。假设两个读者等一份数据，一起醒来；先拿到锁的读者把数据取走，另一个稍后拿锁时，缓冲又空了。若它不重新检查就读取，便会违反条件。

因此等待模板使用 `while`，而不是只检查一次的 `if`。每次恢复后重新验证，条件仍不满足就再次睡眠。

锁负责共享状态的协调，`wakeup` 帮助等待者有机会继续。只有锁没有通知，进程可能一直等；只有通知没有锁，检查与修改可能发生错误交错。两者解决相互配合但不同的问题。

12.8 死锁是一张等待环

A 先持有 L1，再等 L2；B 先持有 L2，再等 L1。两者都不能前进，这叫死锁。画出“谁持有谁、谁等待谁”的箭头，环往往比一大段日志更清楚。

常见办法是统一同时持锁时的获取顺序，缩短持有范围，或先取得引用、释放某把锁，再按合法协议获取另一把。不能因为源码中先后出现几次加锁，就假定这些锁始终同时持有，要检查中间是否已释放。

调度中的 `p->lock` 有特定交接约定，这种例外不允许推广成“持任意自旋锁都可以睡眠”。普通可能阻塞的操作必须符合调用上下文的限制。

12.9 停下来推演

问题 1：单核还会并发吗？ 会。中断和主动让出 CPU 都可以让执行交错；多核只是让同时执行的情况更多。

问题 2：wakeup 是否让对方立即继续？ 它通常先让等待进程变可运行，何时真正执行由调度决定。恢复后条件还可能再次变化。

练习。 用第 12.1 节的表格方式重写空闲链表的“读头、改头、返回页”，构造两个 CPU 得到同一页的错误交错。再圈出必须受同一锁保护的范围。

源码路线：spinlock.c→sleeplock.c→sleep/wakeup→pipe.c。先写出每个锁保护的条件，再核对实现。

第四篇 / 接口、并发与通信

13 文件描述符与管道 把程序连接起来

学习目标：区分 fd、file 与 inode；通过引用理解共享偏移；推演管道读写、阻塞与 EOF。

13.1 fd 是当前进程中的一个编号

程序执行 open 后得到一个小整数，叫文件描述符，英文 file descriptor，简称 fd。它是进程文件表中的索引。内核据此找到真正的打开对象，而不是把所有信息都塞在整数里。

0、1、2 通常约定为标准输入、标准输出、标准错误。它们是当前进程的槽位：fd 1 可以连控制台，也可以被 shell 改为普通文件或管道。因此 write(1,...) 并不必然表示屏幕输出。

术语卡：接口编号只是入口，底层对象才保存类型、权限、偏移和引用等状态。不同进程中相同数字的 fd 可以指向不同对象。

13.2 为什么还要有 file 这一层

进程的 fd 槽位保存指向内核 file 对象的指针。file 表示一次打开实例，保存读写权限、类型和普通文件的当前位置 off。普通文件对象还会关联 inode，inode 保存文件本身的元数据。

先把 inode 理解为“文件身份与存储信息”，第 14 章再展开。此时关键是分清：名字、打开实例与进程编号不是同一个东西。

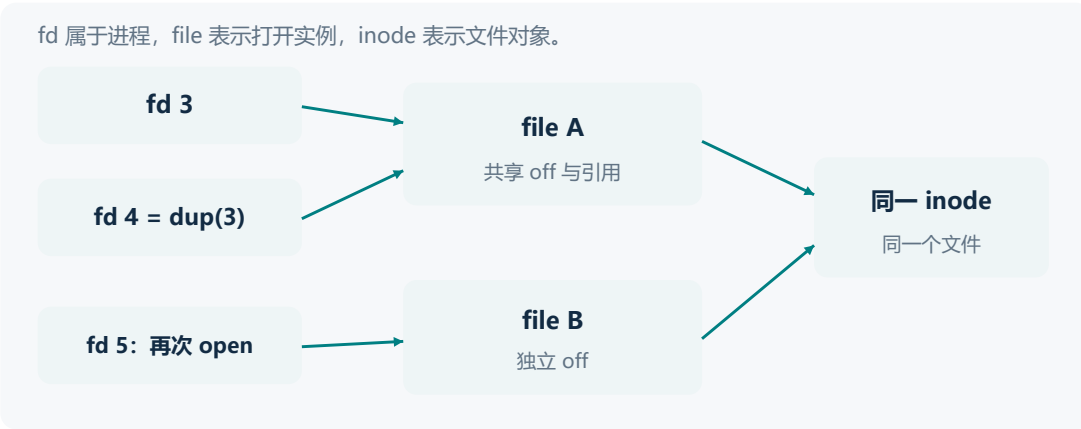


图 13-1 两个 fd 共享一个打开实例，另一次 open 保有独立偏移

两次独立 open 同一个文件，通常取得两个 file，各自从自己的偏移读取。dup 则新增 fd 指向已有 file，所以共享偏移。fork 继承文件表连接时，也增加指向相应 file 的引用。

13.3 用四个字节推演共享偏移

假设文件内容是 ABCD，fd 3 关联 file A，偏移初始为 0。dup 得到 fd 4，但仍指向 file A。从 3 读 1 字节得到 A，偏移变为 1；再从 4 读 1 字节得到 B，偏移变为 2。

如果重新 open 得到 fd 5，它关联新的 file B，初始偏移为 0，从 5 读取仍得到 A。数据文件相同，却有不同打开实例。

操作	得到什么	偏移属于谁
read(3, ..., 1)	A	file A 前进到 1
read(4, ..., 1)	B	同一 file A 前进到 2
独立 open 后 read(5, ..., 1)	A	file B 从 0 前进到 1

引用计数记录还有多少连接持有对象。close 一个 fd 先断开当前槽位，再减少引用。若其他 fd 仍指向同一 file，底层对象还不能因为一个 close 就被销毁。

13.4 管道是内存中的字节缓冲

pipe 创建读端和写端，两者连接到同一段内存缓冲。写者放入字节，读者取走字节。数据不通过磁盘持久保存，管道用于进程之间临时通信。

缓冲有固定容量，像一段首尾相接的数组。累计写入数为 nwrite，累计读出数为 nread，占用量按计数协议理解为 nwrite-nread。真正访问数组时，用计数对容量取余，回到数组中的循环位置。

例如容量为 4，依次写入 A、B、C、D 后满了；读出 A、B 后，前两个位置可以复用写 E、F。逻辑顺序仍是 C、D、E、F，不因为数组索引绕回开头而改变。

13.5 空、满和结束是不同状态

当前条件	读者或写者的行为	为什么
有数据	读者取得部分字节	当前能完成工作
空，但仍有写端	读者等待	将来还可能写入
空，所有写端关闭	read 返回 0	不会再有新字节，这是 EOF
满，仍有读端	写者等待空间	读者可能腾出位置
没有任何读端	写入返回错误等实现规定结果	无人再消费数据

EOF 是 end of file 的缩写，表示输入结束。对管道，它不只是“暂时没数据”，还必须确认所有写端连接都已关闭。管道是字节流，应用需要自行识别消息边界，不能假定一条 write 对应一条等长 read。

等待时采用上一章的条件循环，释放条件锁并正确登记睡眠；对方读写或关闭端点后通知等待者。恢复时重新检查，是因为空、满和端点状态都可能再次变化。

13.6 为什么多留一个写端就会卡住

设父进程创建管道后 fork。父子都继承 r、w。子进程只需要写，却不需要读；父进程只需要读，却仍保留 w。如果子进程写完关闭 w 并退出，父进程自己的 w 还开着，内核仍认为可能有写者。

当缓冲读空，父进程会等未来的数据，而不是收到 EOF。它在等待自己不打算进行的写入。修复时需要关闭父进程多余的 w，子进程也关闭多余的 r。

分析时逐个统计所有 fd 引用，不能只问“写程序是否结束”。fork、dup 以及辅助进程都可能保有写端，最后一个引用关闭才改变最终结束条件。

13.7 shell 怎样完成管道和重定向

`echo hello | wc` 中，shell 的执行分支创建管道并安排两侧。左侧先关 fd 1，再 dup 写端，让最小空闲槽 1 连向管道；右侧先关 fd 0，再 dup 读端。各方随后关闭多余 r、w，再 exec 对应程序。

echo 仍写自己的 fd 1，wc 仍读自己的 fd 0，命令无需专门了解对方。父执行分支也关闭管道两端并等待。这解释了统一 I/O 接口为什么能支持程序组合。

`echo hi > out` 则把输出槽位连到打开的文件。若要覆盖旧文件全部内容，还要检查是否使用 O_TRUNC 等截断标志。只有创建和可写，不自动等同于清除旧文件尾部。

13.8 停下来推演

问题 1：dup 会复制一份文件内容吗？ 不会，它新增 fd 连接到相同 file，连当前位置也共享。

问题 2：管道空时为什么不总返回 0？ 仍有写端时将来还可能来数据，暂时空不等于输入结束。

纸笔练习。 画父、子两张 fd 表，逐项执行 fork、关闭不用端、写字节、关闭写端、读至 EOF。每一步都写出还有哪几份写端连接。若加入一次 dup，继续追踪新增引用何时关闭。

源码路线：file.h→open/dup/close→fileread/filewrite→pipe.c→sh.c。对象关系图应先于复杂调用链。

第五篇 / 文件、设备与命令

14 文件系统
名字怎样找到磁盘上的字节

学习目标：认识目录、inode、磁盘块与位图；理解打开引用和硬链接；从路径一路定位到数据。

14.1 文件名不是数据的存放地址

假设你输入路径 `/a/note`。字符串本身没有告诉内核数据位于哪个磁盘块。文件系统需要先找到根目录，在里面查 a，再在 a 目录查 note，取得文件对象信息，然后定位数据块。

路径是逐层查找的规则。绝对路径从根目录开始，相对路径从进程当前工作目录开始。当前工作目录英文 current working directory，常写 cwd，它属于进程环境，不是整个系统唯一的位置。

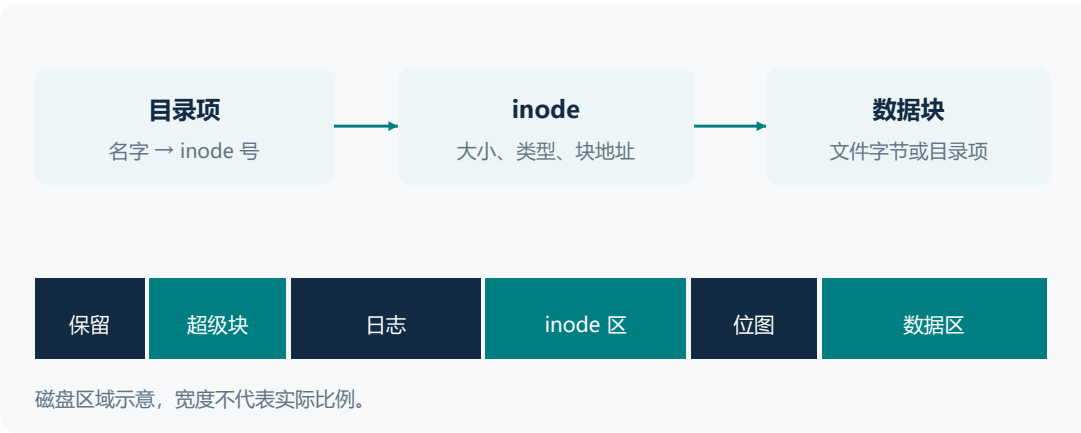


图 14-1 名字、inode 与数据块之间的定位关系

14.2 目录项、inode 与数据块各保存什么

目录内容可以理解为一组记录，每条把名字映射到 inode 号，叫目录项。inode 是索引节点，保存类型、大小、链接数和数据块地址等元数据；文件内容则放在数据块里。

元数据是描述数据的数据，例如“大小为 6 字节”“使用哪几个块”。文件名通常放在父目录项中，不是 inode 只能有一个名字的字段，这使多个硬链接名字能够指向同一 inode。

目录本身也通过 inode 和块保存内容，但普通程序不能随意把字节写进目录，否则可能破坏名字、引用和分配关系。创建、链接和删除应通过受控接口完成。

> 术语卡：磁盘块是文件系统处理存储的固定单位；内存页是地址映射与分配单位。xv6 基线磁盘块常为 1024 字节，内存页为 4096 字节，二者不要混用。

14.3 磁盘上为什么还要有超级块和位图

超级块记录整个文件系统的规模与各区域位置，相当于布局说明。inode 区存放文件元数据，数据区存放文件内容，位图用二进制位标记哪些块已经分配。日志区域帮助故障后恢复，下一章专门讨论。

位图中一个位表示一个块是否使用。分配时查找空位并置为占用，释放时清回空闲。如果位图说空闲，某个文件却仍使用该块，下次分配就可能把同一存储交给两个文件。因此各结构之间也有不变量。

mkfs 在宿主机创建这些初始布局；xv6 挂载自己的镜像后，按同样约定读取。文件系统必须遵守磁盘格式，不能把结构体随意改字段却仍期待旧镜像正确解释。

14.4 内存 inode 为什么多出 ref 和锁

磁盘 dinode 主要保存持久字段。内存 inode 还要记录当前有多少内核路径持有它、元数据是否已加载，以及怎样协调并发访问。因此两种结构相关但不完全相同。

iget 找到或占用内存槽位，并取得引用；ilock 取得对内容的访问资格，必要时从磁盘加载；iunlock 释放内容锁；iput 归还引用，并在满足最终回收条件时处理数据释放。

引用使对象在查找和使用期间不会被当作无人持有的槽位复用，锁则协调可变内容的访问。对象活着并不等于你可以无锁修改它；拿到锁也不能替代持有正确引用。

14.5 删除名字后，已打开文件为什么还能读

对普通文件，nlink 反映文件系统中的硬链接关系。ref 反映内核中还有多少持有引用。open 经 file 对象持有 inode，unlink 只删除一个目录名字并更新链接关系。

假设 note 只有一个名字，你先 open，再 unlink。名字没了，后来的路径查找找不到它，但现有 fd 仍通过 file 指向 inode，读取可以继续。最后一个打开引用关闭，且链接数为零时，数据才达到回收条件。

动作	名字是否存在	已打开 fd 是否仍可达对象
创建并打开	存在	可达
删除最后名字	不存在	仍可达
关闭最后引用	不存在	不再可达，可按规则回收

目录链接数有专门的维护约定，不完全等于简单统计目录里的任意名字。初学先掌握普通文件例子，再看 mkdir、unlink 的额外更新。

14.6 一步步查找 /a/note

内核先取得根目录引用并锁住内容，检查它确实是目录；查目录项 a，取得下一 inode 引用；释放旧目录的内容锁和引用，再对新目录重复过程，最终查到 note。

每一段交接都需要保住对象引用，但一般不把整条路径的所有目录同时锁住。若中途发现名字不存在或对象不是目录，要释放已取得的锁和引用再返回失败。

namei 用于查到目标，nameiparent 用于停在最后一级父目录。例如创建 note 之前目标还不存在，需要先取得 a，之后才能在它的目录内容中加入一个新名字。

14.7 文件偏移怎样变成块号和块内偏移

若磁盘块为 1024 字节，读取文件偏移 1500，就有 1500 = 1 × 1024 + 476。内核先找该文件第 1 个逻辑数据块，再从块内偏移 476 取内容。这里的逻辑块号不是整个磁盘块号，还需由 inode 的块地址映射。

基础 inode 有 12 个直接块地址，再有一个间接块地址。间接块存一组数据块号，帮助容纳更多文件内容。1024 字节块中若每地址 4 字节，可放 256 个地址，因此最多有 12+256 个数据块，容量约 268 KiB。

假设文件需要 13 个数据块，前 12 块由直接地址指向，第 13 块需要额外的间接索引块。索引块自身也占存储，不能只数用户数据。截断时既要释放数据块，也要释放不再需要的索引块。

14.8 从 read 回到用户数组

read 先从 fd 找到 file，再从 file 找 inode。readi 限制文件读取范围，bmap 找到对应磁盘块，bread 从缓存取得该块，必要时让驱动读盘。最后通过用户复制路径把数据交到用户缓冲。

每层解释的编号不同：fd 是进程槽位，inode 号是文件身份，逻辑块号是文件中的位置，磁盘块号是文件系统中的块位置。把这些层次分开，是理解文件系统最有效的方法之一。

缓存可以让某次读取不必真正发磁盘请求。文件末尾之后的内容也不能当作有效字节返回，readi 应按文件大小限制范围。

14.9 停下来推演

问题 1：inode 为什么不只存一个文件名？ 多个目录项可以给同一文件不同硬链接名字，名称属于目录关系，inode 表示被指向的对象。

问题 2：有引用就可以随意修改 inode 吗？ 不可以。引用保证对象还被持有，内容锁保证并发访问按规则进行。

练习。 画出 `/a/note`、一个已打开 fd、file、inode、两个数据块。先删除目录名字，再关闭 fd，分别划掉失去的连接。最后计算 1500 字节位置的逻辑块号与块内偏移。

源码路线：fs.h→iget/ilock/iput→namex→readi→bmap→bread。第一次可以只追一个已有小文件，创建与删除放在之后。

第五篇 / 文件、设备与命令

15 写了一半突然断电 日志怎样恢复一致

学习目标：区分运行时互斥与崩溃一致性；解释缓存和 pin；用提交阶段推演故障后的结果。

15.1 为什么一次写文件涉及多个地方

创建并写入新文件，可能更新 inode、父目录、数据块和分配位图。如果只有目录项写好了，文件身份还没正确建立，重启后就可能出现相互矛盾的信息。若块被占用却没有文件能找到，也会浪费空间。

即使没有两个 CPU 同时修改，断电也可能发生在任意两次磁盘写之间。因此锁只能协调运行中的执行者，不能自动让多块更新在掉电后完整留下。

崩溃一致性关注重启之后结构是否仍符合约定。它与“用户的每一次写入都已立刻永久保存”相关但不相等。先学会恢复一组已提交修改，再讨论用户看到的持久化语义。

15.2 buffer cache 为什么既缓存又同步

buffer 是一个磁盘块在内存中的副本，cache 是缓存。相同设备和块号应找到统一的缓存实例，避免两条内核路径各自修改互不相见的副本。

缓存表的锁保护查找、引用和回收等元数据，单个 buffer 的睡眠锁保护内容。bread 返回已经取得内容锁的块，brelse 表示当前调用者结束使用。若内容已经有效，读者不必每次都访问设备。

事务中某些块即使暂时没人使用，也不能被回收换成另一块，否则准备提交的内容会丢失。pin 表示把缓存对象保留下来；它保证留存，不等于给调用者独占内容。留存与访问同步应分开理解。

15.3 先留一份可重做的副本

redo 的意思是重做。物理 redo 日志保存修改后的块副本，并记录它们应该安装到哪个正式位置。这样，即使正式位置只更新了一部分，重启后仍有完整副本可以再次覆盖。

日志头保存本批次目标块号和数量。它还承担关键承诺：非空有效头表示这批修改已经提交，恢复必须把它们安装好；空头表示没有这样的待恢复承诺。

> 术语卡：提交是宣布某组修改进入约定的可恢复状态；安装是把日志副本写到正式位置。提交和安装是两个相邻但不同的阶段。

15.4 四步提交要按什么顺序

第一步，将缓存中最终块内容写入日志数据区。第二步，写日志头，宣布这组副本与目标位置有效。第三步，按头记录把副本安装到正式位置。第四步，清空头，表示恢复任务已经完成。

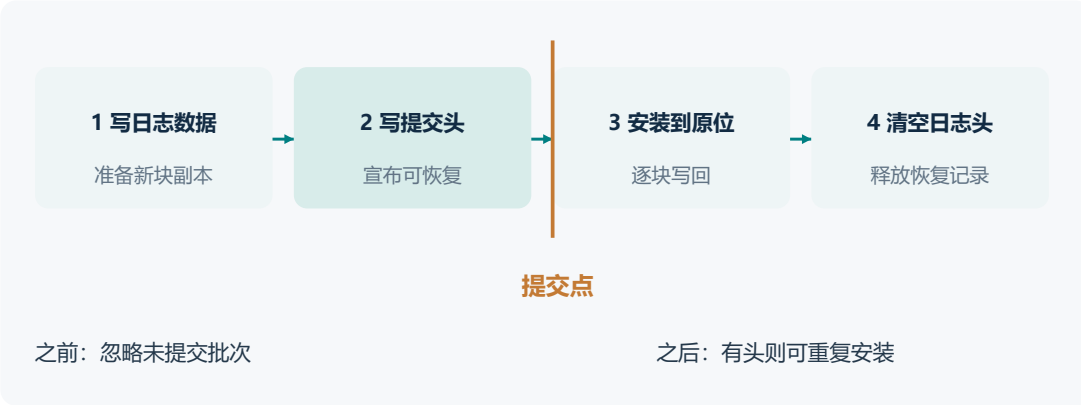


图 15-1 日志提交的准备、承诺与安装

顺序的理由可以逐条检查：如果头先写、副本还未完成，恢复会相信不存在的完整数据；如果安装前先清头，安装中断电就失去修复依据；若最后没有清头，下次启动还会重复识别旧事务为待恢复工作。

同一缓存块在一个批次中多次修改，通常不必记录多个副本，提交时采用最终内容。日志描述本批次的新块状态，不是逐行记录 C 程序执行过程。

15.5 亲手推演五个断电位置

断电时机	重启时的头	恢复办法	最终结果
副本只写一部分	空	忽略未提交批次	保持此前提交状态
副本完成，头未写	空	仍不安装该批次	保持此前提交状态
头已写，未安装	非空	安装所有记录块	得到新状态
安装了一部分	非空	重复安装所有记录块	补齐新状态
安装完成，头未清	非空	再安装一次	仍是同样新状态

重复安装为什么没问题？因为把同样内容写到同样位置，结果不变，这种性质叫幂等。恢复期间再次断电，下次仍可按照非空头重做，不必猜上次停在哪一个块。

这段推演使用 xv6 教学设备模型依赖的写入顺序和块写入假设。真实存储的易失缓存、重排、撕裂写与故障需要进一步协议，不能直接从本表得出所有硬件都已获得完整掉电保证。

15.6 为什么 begin_op 要预留空间

日志区域容量有限，一次操作可能修改多个块。begin_op 不仅看已经占用多少，还要为尚未完成的并发操作预留最坏情况空间；不够时让调用者等待。否则多个操作都觉得“还有一点空位”，合在一起却溢出。

end_op 表示一个操作完成。最后一个 outstanding 操作结束时，才由相应路径提交这一批。outstanding 可以理解为“已进入、尚未结束的操作数”。多个系统调用因此可能共享一个提交批次。

大型 write 还可能拆成多个日志操作，避免超出容量。因此任意大的用户 write 不保证在崩溃后整体全有或全无；一次系统调用也不能机械等同于一个独立磁盘事务。

15.7 一致性、隔离与持久性分别看什么

本章主要解释故障后结构恢复。运行中是否能看见别的操作中间状态，要结合各层锁与接口语义分析；某次返回是否表示数据已永久落盘，也需要知道所在批次是否真正提交。

xv6 日志可包括经 log_write 纳入的元数据和相关数据块，不只是元数据日志。但某个操作返回时可能还有并发操作，让该批次尚未提交。因此“结构可以恢复”不等于“我刚写的每字节都一定不会丢”。

这些限制并不否定日志的价值。它让原本多处不匹配的写入，能通过一个明确提交标志和重复安装过程恢复到合理状态，形成可以推演的故障协议。

15.8 停下来推演

问题 1：为什么给所有文件操作加锁也不能解决断电？ 锁在内存和执行者之间生效，断电后不再帮助完成尚未写完的磁盘更新。

问题 2：为什么不能看见日志数据就全部重放？ 未写有效提交头的副本可能不完整，也可能只是尚未承诺的一批准备工作。恢复必须依约判断提交。

纸笔练习。 让一次更新涉及两个正式块 A、B，日志中保存 A新、B新。分别在写头前、写头后、只安装 A 后断电，写出重启时怎样处理，检查最终是否会留下永久的半新半旧组合。

源码路线：bio.c→log_write→begin_op/end_op→commit→recover_from_log。真实故障实验应使用可丢弃镜像副本，并在重启时保留同一镜像。

第五篇 / 文件、设备与命令

16 设备驱动 内核怎样等到硬件完成

学习目标：理解设备寄存器、MMIO 和请求队列；从提交到中断完成走一遍；分清进程等待与中断处理。

16.1 为什么不能像调用 add 那样直接完成磁盘读

CPU 做一次加法通常很快，磁盘请求却涉及另一个设备的工作。内核先描述想读什么，设备按自己的时序执行，再报告结果。设备并不直接理解 C 函数或文件路径名。

驱动程序负责把上层请求转换成设备协议：初始化设备，准备请求，通知执行，接收完成，再把结果交给等待者。它是内核与具体设备之间的适配层。

例如文件系统已经把路径解析成磁盘块号，驱动只需要完成块读写。让驱动再去理解 `/a/note`，会把文件系统规则与硬件协议混在一起。

16.2 设备寄存器与普通内存有何不同

设备提供一些控制与状态位置，例如“是否可接收数据”“提交哪个队列”“确认中断”。这些位置叫设备寄存器，与 CPU 内部通用寄存器不是同一组对象。

MMIO 是 memory-mapped I/O，即内存映射输入输出。设备寄存器占据特定地址范围，内核通过类似读写内存的指令访问。但背后不是普通 RAM：写某位置可能启动设备，读某位置可能清除状态。

因此读写次数、宽度、顺序与屏障都可能有意义。把同一个寄存器“为了看清楚再读一次”，也可能改变设备行为。驱动要按协议操作，不能只套普通变量的经验。

16.3 轮询和中断是两种等待办法

轮询表示 CPU 反复读取状态，检查设备是否完成。中断表示设备在需要处理时通知 CPU。前者直接但可能消耗许多检查时间；后者允许进程睡眠，把 CPU 留给其他任务。

轮询并非总是不合适。启动早期还没有完整调度环境，紧急输出也可能不能依赖正常等待路径；这时同步轮询可以很实用。正常耗时不确定的 I/O 则常适合用队列、中断与睡眠合作。

要区分三个角色：提出请求的进程、执行请求的设备、处理完成通知的中断代码。提出请求的进程可以暂停，但设备和其他 CPU 工作仍在推进。

16.4 描述符与环形队列怎样传请求

VirtIO 是一种常用于虚拟化环境的设备接口体系。xv6 通过 VirtIO 磁盘驱动访问模拟硬盘。描述符是记录地址、长度与方向等信息的小结构，可以串成链，描述请求头、数据缓冲和结果状态。

驱动把可处理的描述符编号发布到 available ring，即可用环；设备完成后把结果放到 used ring，即已用环。ring 表示队列位置按容量循环使用，逻辑上的先后顺序由索引和协议维持。

可以把描述符看作货运单，但货运单上的地址必须是设备按协议能访问的位置。CPU 虚拟地址、物理地址与设备协议所用地址不能不加区分地互换。内存共享与顺序要求也需按协议满足。

16.5 一次缓存未命中的磁盘读

第一步，bread 发现缓存内容尚未有效，调用磁盘驱动。第二步，驱动取得队列资源，建立描述符链并通知设备。第三步，请求线程在条件循环里等待完成，按协议释放条件锁并睡眠。

第四步，设备写好数据与完成信息，产生中断。第五步，virtio_disk_intr 读取已用队列，更新请求完成状态，唤醒等待者。第六步，等待者重新运行、检查完成条件，释放请求资源，让 bread 返回已锁定的缓存块。



图 16-1 异步磁盘请求：进程等待，设备完成，中断唤醒

若缓存已经有效，bread 可以省去实际设备请求。读盘完成也不代表用户数组已填好，数据仍在块缓存，上层还要沿 inode 读取与用户复制路径交给进程。

16.6 PLIC 为什么要领取和完成

PLIC 是平台级中断控制器，把多个外部设备中断组织起来，按配置向 hart 提供待处理编号。claim 表示领取一个中断请求编号，complete 表示通知控制器这次处理结束。

内核的 devintr 判断外部中断后，从 PLIC 取得编号，再调用 UART 或 VirtIO 对应处理函数，最后 complete。设备本身还可能有自己的确认寄存器，设备确认和控制器完成是不同步骤。

时钟中断通常有自己的体系结构路径，不能认为所有中断都经 PLIC。具体 timer 配置随代码与硬件扩展变化，阅读时结合 start.c 与 trap.c。

16.7 按一个键，为什么 shell 不一定立刻处理

UART 是串口设备。用户在 QEMU 终端输入字符，模拟串口产生外部中断；uartintr 取出字符，console 层处理回显、退格和输入缓冲。缓冲到完整行、达到特定条件或遇到 EOF 控制字符后，读取者才可能取得完整可用输入。

因此，字符已经到达设备与 shell 已经读完命令行是不同阶段。键盘输入路径也有设备状态、缓冲条件、唤醒与进程恢复，结构上与磁盘等待有相通之处。

16.8 中断处理为什么应尽快返回

中断发生时，普通执行路线被暂时打断。处理函数应主要确认状态、取出完成项、更新共享条件并唤醒等待者，不能随意调用会睡眠的函数。复杂工作适合放到恢复后的普通内核路径。

如果驱动持有自旋锁等待完成，而完成中断也要同一锁，设备虽已完成，内核仍可能死锁。睡眠和条件锁协议因此是驱动正确性的一部分，而不是只在管道中才有用。

经典 xv6 为保持代码简洁，某些设备错误直接 panic，没有完整超时和错误恢复体系。理想驱动应把错误可靠记录、通知等待者并安全回收资源，但不能把这种理想直接写成当前代码已具备的功能。

16.9 停下来推演

问题 1：设备发中断后，等待者是否自动出现在下一条 CPU 指令？ 不是。中断处理先改变完成条件并唤醒，调度器以后选择它，它还要重新检查状态。

问题 2：驱动为什么只处理块请求，不解析路径？ 路径属于文件系统抽象，驱动负责具体设备协议。层次分开才能分别修改和验证。

练习。 在一页纸上画三条泳道：请求进程、设备、中断处理。放上“提交、睡眠、设备完成、更新条件、唤醒、继续”六个动作，检查等待者继续前是否已经确认完成。

源码路线：devintr→PLIC→virtio_disk_intr，再反向看 virtio_disk_rw 如何发布和等待。串口路径可作为第二个对照例子。

第五篇 / 文件、设备与命令

17 shell 与 exec 一行命令怎样真正运行

学习目标：连接命令解析、进程和 fd；理解 ELF 与参数栈；完整解释 cat README | wc 的运行。

17.1 提示符来自一个普通用户程序

内核不直接理解 ls、管道或重定向语法。shell 是读取命令行、解释符号并组合系统调用的用户程序。init 先建立控制台输入输出，再创建 shell 执行分支，自己负责等待与必要的重新启动。

命令语法和交互策略于是可以放在用户空间。shell 出错时，理想情况下损坏范围限于该进程，init 还可以再次启动它，无需内核把所有交互逻辑都包含进去。

17.2 先把命令变成树

对 `cat README | wc`，解析器识别左侧命令、竖线和右侧命令，形成一个管道节点，下面连着两个简单执行节点。执行器再按节点类型安排动作。

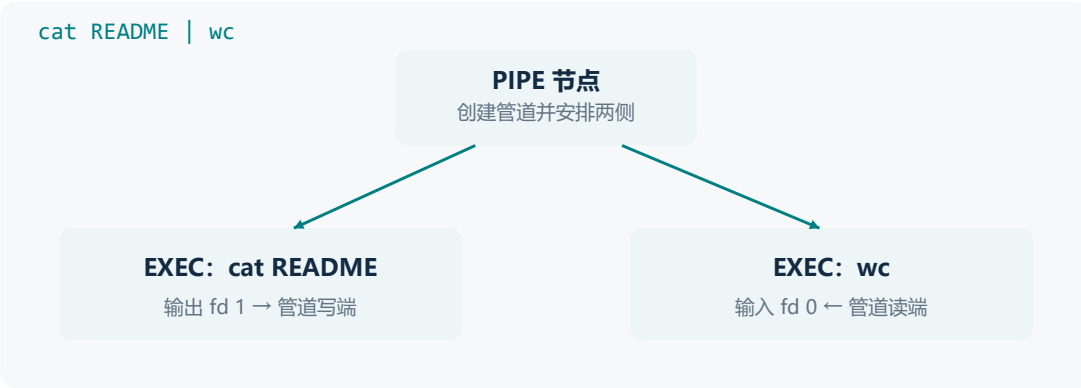


图 17-1 一行管道命令对应的简化命令树

简单节点负责 exec，重定向节点先调整 fd，管道节点创建连接并安排两侧，顺序与后台节点各有等待策略。这种表示把“字符串怎么理解”和“进程怎么执行”分成两个问题。

实际 xv6 shell 可能使用额外管理进程递归执行命令树。因此分析进程总数时，要按源码每个 fork 的实际位置画图，不能仅看有几个命令名。

17.3 为什么通常先 fork，再 exec

fork 创建新进程，exec 替换当前进程的运行内容。交互 shell 若直接 exec ls，原来的解析循环就被替换，ls 结束后旧 shell 不能自动回来。

通常先 fork，让父进程保留命令循环；子进程准备重定向与管道连接，再 exec 对应程序；父进程 wait 回收后继续显示提示符。成功 exec 不返回旧调用点，失败才返回错误。

打开文件的连接在 exec 过程中按 xv6 约定保留，所以子进程可以先把 fd 1 接到 out，再装载 echo。echo 本身仍只需要知道向标准输出写字节。

`cd` 需要特别处理，因为下一条命令要使用交互 shell 自己的新 cwd。若只在临时子进程中 chdir，父 shell 仍在原目录。这个例子让“进程各自持有什么状态”直接影响命令行为。

17.4 ELF 告诉内核怎样摆放程序

ELF 是一种可执行文件格式，包含头部与若干段的描述。装载器根据段信息知道：从文件哪个位置取字节，放到哪个用户地址，内存要多大，允许读写或执行什么。

例如一个段文件中保存 100 字节，而运行时需要 160 字节，那么后面的 60 字节可作为零初始化区域，不需要文件实际保存它们的全部内容。BSS 是常见的零初始化区域名称。

exec 要检查文件格式、范围、对齐、文件大小与内存大小关系及算术溢出，再分配页面和装入内容。它不能把任意文件从头到尾当成可以执行的原始指令，更不能盲目信任文件里的地址数字。

17.5 argc 和 argv 怎样到新程序手中

shell 以参数数组表示命令。例如 cat README 对应两个字符串 cat、README，加一个空指针作为 argv 结束标志；argc 为 2。新程序不能继续依赖旧 shell 或旧地址空间中的字符串位置。

exec 把字符串复制到新用户栈，再放入指向这些字符串的新用户地址数组。栈指针对齐后，用户入口得到 argc 和 argv。经典 xv6 路径通过返回机制把 argc 放入 a0，并准备 a1 指向 argv。

栈上对象	保存的内容	为什么要新建
字符串区	cat 和 README 的字符与结尾零	旧地址空间不再保留
argv 数组	指向新字符串的用户地址	地址必须在新环境有效
初始 sp	满足调用约定的栈位置	入口代码需要正确初始状态

栈旁还可设置用户不能访问的保护页，帮助发现越界。实现懒分配时不能把保护页故障误判为“这里缺了一页，应补成可写”。

17.6 为什么先准备新家，再放弃旧家

exec 可能因为文件损坏、段范围非法、内存不足或参数太多失败。若一开始就销毁旧地址空间，失败后进程连返回错误的环境都没有。

因此先建立临时新页表、程序段和栈，全部成功后再更新当前进程的页表、大小、入口与 sp，释放旧空间。失败则清理临时对象，保留旧程序处理错误的能力。

这是一种提交式资源替换，帮助保证内存中操作失败后的合理状态。它与上一章磁盘日志的断电恢复不是同一个保证，不能因为都叫“提交”就认为机制完全相同。

17.7 完整走一次 cat README | wc

第一阶段是准备执行：shell 读取整行，构造管道树，创建管道和两侧执行分支。左侧将输出接到管道写端，右侧将输入接到读端，各方关闭多余引用，再分别 exec cat 与 wc。

第二阶段是数据流动：cat open README，read 经 file、inode 与块缓存取得内容；缓存缺失时提交磁盘请求并等待。完成后，copyout 把字节放进 cat 的用户数组。cat 再 write，copyin 从它的数组取得字节，放进管道。

第三阶段是消费和结束：wc 从自己的 fd 0 读取管道，可能在空时睡眠，读到字节后统计。cat 结束并关闭最后写端，wc 读尽剩余字节后得到 EOF，输出结果并退出。等待者回收相应进程，交互 shell 继续。

这不是一条毫不停顿的直线。磁盘未完成、管道为空或已满时，进程都可能暂停；数据也可能分成多轮 read/write。正确地图要同时包含调用关系与等待条件。

17.8 停下来推演

问题 1：exec 为什么不改变 PID？ 它替换当前进程内容，不是新建另一份进程身份。文件连接和父子关系得以按约定保留。

问题 2：命令不存在时为什么还能看到下一次提示符？ 子进程 exec 失败后退出，父 shell 保留了解析循环，wait 回收后继续。

综合练习。 分别为 ls、echo hi > out、cat README | wc 画三样东西：命令树、进程树、fd 图。再给每条 read/write 标出可能访问的对象和可能睡眠的位置。

源码路线：init.c→sh.c 主循环→parsecmd/runcmd→sys_exec→exec.c。每次先研究一种命令节点。

第六篇 / 独立观察与验证

18 调试
让每一次观察回答一个问题

学习目标：分类错误；构造最小复现；理解日志、断点、调用栈与回归测试提供的证据。

18.1 先说清楚“不工作”是什么意思

不能编译、不能启动、运行后 panic、返回错误、一直无输出，是不同现象。构建问题通常先看声明、符号和 Makefile；启动问题先看初始化与地址；行为问题再沿具体请求找状态和等待。

panic 是内核在发现严重异常或不变量被破坏时停止的诊断路径，不是普通用户函数的错误返回。卡住也不一定是死循环，可能是正确睡眠却没有任何人再改变条件。

调试第一步应写预期与实际。例如“所有写端关闭后，读者应返回 0；实际一直睡眠”，比“pipe 有问题”更能引导下一次观察。

18.2 什么是最小复现

最小复现是保留触发条件、删掉无关动作的小程序或操作序列。复现 EOF 问题可以只做 pipe、fork、关闭端点、少量写入、读至 EOF、wait，不必夹杂创建目录和大量打印。

“最小”不意味着删到看不见问题。若故障依赖多核交错、跨页地址或父进程额外写端，这些条件必须保留。每删一项后都要确认原现象仍可复现。

记录源码版本、CPU 数量、构建和运行命令，才能让别人处于同样条件下验证。只记录一句“偶尔会卡”，很难区分代码差异与时序差异。

18.3 日志与断言各回答什么

日志记录“执行到哪里”“当时字段是多少”。断言则在不变量不成立时立即报告，例如某页不应被重复释放。好的日志围绕一个假设，好的断言围绕一个必须保持的关系。

例如怀疑多余管道写端，可记录每次创建、复制和关闭引用，重建连接图。单纯在每个函数入口打印一次名字，输出虽多，仍未回答哪个写端还存在。

打印本身会花时间、使用串口与锁，改变执行交错，所以 bug 可能在加日志后暂时消失。应限制次数、聚焦对象，并在去掉临时日志后重新验证，不把观察扰动当成修复。

18.4 断点、单步和调用栈怎样帮助理解

断点使程序执行到某个位置暂停，便于查看状态；单步让你观察下一条源码或机器指令；调用栈显示当前一层层函数是怎样来到这里的。GDB 是常用调试器，QEMU 可以提供远程调试接口。

在宿主机一个终端运行 make qemu-gdb，使目标按配置等待调试连接；另一个终端使用支持 RISC-V 的 GDB 加载 kernel/kernel，并按当前 Makefile 与生成的 .gdbinit 连接。工具名、端口与自动连接配置要以所用版本为准。

连接成功后，可从最简单服务开始：

GDB / 连接成功后执行

```
break syscall
break sys_getpid
continue
```

断住后，用 bt 查看调用栈，用 info registers 看机器寄存器，用 print 查看 C 对象，用 x 命令按指定格式检查内存。第一次只追 getpid 的入口、分派和返回值，不同时追设备中断与所有进程。

18.5 查看寄存器时别看错保存位置

进入 C 处理函数后，硬件 a0 可能已被函数调用重新使用，用户原参数则在 trapframe 字段中。若要观察用户传来的值，应找正确保存对象，而不是一直盯着当前寄存器。

用户 PC 与内核 PC 也属于不同程序。内核符号可以借 kernel/kernel 及反汇编辅助定位，用户故障地址应对应相关用户可执行文件。拿内核符号解释一个用户地址，可能得不到有意义结果。

故障日志中的原因、PC、故障地址和当前大小要组合阅读。store page fault 只是说明写访问不能完成，还要判断地址范围、权限、映射以及是否支持合法修复。

18.6 将“卡住”拆成几种可观察状态

观察到什么	下一步先查什么	可能要验证的关系
进程睡在管道上	数据和端点引用	为空还是已结束，谁可能继续写
CPU 在锁循环中	持有者与锁顺序	是否存在等待环
磁盘已提交却未完成	队列与中断	设备是否完成，通知是否被处理
进程可运行但很久未运行	调度、中断与进程锁	是否仍有合法选择机会
用户坏地址导致内核停止	用户复制与异常边界	是否直接解引用不可信地址

这些是观察入口，并非立即定罪。一个睡眠进程可能完全正确，关键是条件是否还能改变、改变者是否有机会继续、通知是否按协议送达。

18.7 测试要针对你声称修复的关系

修复文件引用泄漏，就重复 open/close，检查是否最终耗尽；修复 EOF，就测试多份端点、空传输、正常传输和不同关闭顺序；修复内存复制，就测试跨页、权限与非法范围。

涉及并发时，增加单核与多核对照、重复运行，并用锁与状态推理补足未枚举交错。usertests、forktest、stressfs 等测试也提供了行为规格，先读一个与你目标有关的测试即可。

回归测试指修复后重新验证曾出错的行为和相关旧行为，避免修好一处又破坏另一处。报告只写实际运行的检查，未运行的应明确列为待验证。

18.8 停下来推演

问题 1：加日志之后不再出错，是否能证明修好？ 不能。日志可能改变时序，根因仍存在。需要解释不变量并在正常配置下重新验证。

问题 2：所有测试通过是否证明绝无 bug？ 不能。它说明覆盖到的行为符合预期，仍应考虑未覆盖输入和并发交错。

练习。 写一份六行记录：最小操作、预期、实际、第一个异常证据、相关不变量、回归结果。使用纸笔注入“父进程忘关写端”的错误，也能练习完整证据链。

源码路线：一个相关测试→最小操作→系统调用→对象状态与锁。每次观察前，先写它能支持或推翻哪个假设。

第六篇 / 独立观察与验证

19 循序完成一个小型操作系统项目

学习目标：把概念变成有限范围的修改；先定义接口和资源协议；用正常、失败和并发情况验证设计。

19.1 第一个项目可以很小

刚学完时，不必立刻改调度策略。先写一个用户命令，打印自己的 PID，再调用已有接口完成一个小任务，可以练习构建、用户头文件、执行与观察。

xv6 用户程序 / 按注释区分完整程序与片段

```
// xv6 用户程序，可作为单独源文件加入用户构建。
#include "kernel/types.h"
#include "user/user.h"
int main(void) {
  printf("my pid = %d\n", getpid());
  exit(0);
}
```

把产物加入 Makefile 的 UPROGS，重建镜像并执行。随后沿 getpid 路径解释输出来自哪里。这个项目虽小，却要求你把源码、构建、进程与系统调用连起来。

19.2 再增加一个只读内核接口

完成第 11 章 mypid 接线练习之后，可以设计只返回当前进程少量信息的接口。先规定一个专用用户结构体，只放 PID、大小等明确公开的字段，规定字段宽度、返回值与失败行为。

内核先收集必要值，初始化整个输出对象，再受控复制给用户。不要把 struct proc 原样复制，不暴露内核指针和锁。接口容量不足、非法地址、跨页失败和复制副作用都应有约定。

测试先覆盖正常查询，再传未映射地址和不合法长度。若接口中途失败，也应保证没有泄漏临时页或引用。用户看见的结果与内核内部结构之间，需要一条清楚边界。

19.3 进阶一点：调度次数统计

可以在调度器选择进程、把它从可运行改为正在运行的位置增加计数。先定义“次数”是被选中的次数，不是运行秒数、时钟 tick 或指令数。

再规定字段由谁写、由哪把锁保护，fork 后初始化为何值，退出前如何读取。若只加一个整数却没考虑多核和读取协议，就把原来简单接口变成了竞态来源。

统计进程列表还需要定义快照语义。逐项读取并不天然代表同一时刻的全系统状态，某些字段也未必由 p->lock 统一保护。必须按实际字段的同步协议读取，再向用户解释结果的适用范围。

19.4 文件系统与 shell 项目怎样选范围

shell 项目可以增加一个语法或内建命令，但要先画命令树与 fd 图，确认改变目录等状态应作用于哪个进程。文件系统查询项目则要定义统计是瞬时近似还是受一致性约束的结果，并检查缓存、位图、锁与事务边界。

项目	新增的主要难点	动手前应会什么
用户查询命令	构建与接口调用	第 5、11 章
当前进程只读信息	用户复制与失败语义	第 8、11 章
调度次数统计	共享字段与锁	第 9、12 章
shell 行为扩展	进程、命令树和 fd	第 13、17 章
文件系统空间查询	块、缓存与统计语义	第 14、15 章
写时复制 fork	共享页、权限、故障与回收	完整内存与并发模型

每次只新增一种主要约束，方便判断错误来自哪里。代码越多不代表理解越深，能把小范围设计讲清楚更重要。

19.5 进阶：写时复制为何牵涉这么多路径

写时复制简称 COW，意思是先共享内容，等需要写入时再分离。fork 时父子暂时共享物理页，对原本可写的用户页撤销 W 并标记为 COW，同时增加物理页引用。

某一方首次写入，产生故障。内核确认这是合法 COW 后，分配新页、复制旧内容、更新当前映射、减少旧页引用，并按要求刷新翻译。原本只读的代码页不能因为写故障就被当成 COW 改可写。

最后一个引用消失，物理页才能归还。fork 失败、exit、exec、解除映射和 copyout 都要遵守同一协议。特别是内核通过自己的映射写物理页，可能不会触发普通用户写故障，因此 copyout 也要识别 COW。

这个项目的核心不是会 memcpy，而是所有拥有和释放路径都认同同一份引用规则。基础机制尚未稳固时，可以先在纸上推演，等能够解释每条路径再动手。

19.6 设计说明先于代码

写代码前，用一页纸回答六个问题：用户将看到什么新行为；对象由谁创建和释放；哪些字段共享；什么不变量必须成立；每个失败位置已经取得什么资源；哪些测试能够验证这些声称。

若要持锁读取对象，再复制给用户，需特别检查复制是否可能分配或阻塞。不要长期持自旋锁调用不适合的 copyout 路径。可以按协议取得有界内核快照，释放相应锁，再向用户复制，并明确允许的陈旧程度。

说明应与最终代码一致。项目过程中推翻旧设计很正常，但最终文档不能仍描述已经放弃的机制。

19.7 完成项目的验收方法

先展示正常行为，再展示非法输入、资源不足或并发变化时会怎样。附上实际版本、配置、命令、结果和已知限制，让别人可以复现。

调试记录至少留下一次完整推理：最初怀疑什么，哪个观察支持或推翻它，最后修复保护了哪条关系。只说“改完以后能跑”无法说明失败路径是否正确。

最后重画两张全链路图。第一张是 cat README | wc 的进程、内存、文件与设备路径；第二张是 fork 到 exit/wait 的对象创建、共享与回收路径。每个箭头都应能说出对应状态和责任人。

19.8 停下来推演

问题 1：为什么接口返回结构体应清零？ 除了明确字段，结构体可能有填充字节；未经初始化的内容不应泄漏给用户。接口还应避免暴露内部指针和布局。

问题 2：为什么不能只实现 COW 的用户缺页分支？ 内核向用户页写入的路径也可能修改共享物理页，不一定通过同一用户故障入口。所有修改和释放路径都要纳入协议。

起步任务。 先完成第 19.1 节命令，能解释每个 include、构建登记、getpid 与 exit，再决定下一项。把尚不清楚的术语变成具体问题，回到对应章节验证。

OPERATING SYSTEMS / XV6

结语 从会写语句，到理解运行中的系统

你从变量、循环和函数出发，走到了 CPU 的执行位置、程序的内存布局与运行实例。随后看到内核如何分配资源、让进程交错运行、用页表保护地址，再通过系统调用与中断连接用户和设备。

这些机制不是互不相干的名词。read 需要检查用户内存，可能让进程等待磁盘；等待依赖状态、调度和锁；设备完成通过中断改变条件；恢复后数据进入用户数组，文件引用又要在退出时正确关闭。一个简单动作，正是这些小规则共同成立的结果。

继续学习时，仍可以使用同样的顺序：先找一个现象，明确对象，画出状态变化，再阅读实现。如果某个英文缩写让你停住，先问它在替谁保存什么、让谁能继续、保护哪条边界。名称就会重新落回具体工作。

理解并不要求一次记住所有细节。能用一个小例子讲清楚，能预测一个变化的后果，能设计观察来核对预测，就是实实在在的进步。把这些习惯带到下一次阅读中，你会逐渐拥有自己解释系统的能力。

OPERATING SYSTEMS / XV6

附录 A 按学习顺序查术语

术语	直接含义与回忆线索	章
CPU	执行机器指令的处理器，不直接理解 C 源码	1
bit / byte	二进制位 / 字节；8 位组成 1 字节	1
内存 / 持久存储	当前工作内容 / 需要在断电后保留的文件	1
寄存器	CPU 内部保存少量当前状态的位置	1
地址 / 值	位置编号 / 位置中保存的内容	1、4
PC	程序计数器，指示执行位置	1
并发 / 并行	任务交错推进 / 同一时刻多个执行者工作	1、12
编译 / 链接	翻译目标代码 / 连接符号与各部分布局	2
符号 / 库	函数等对象的标识 / 可复用实现集合	2
装载	按文件描述准备地址空间与初始执行环境	2、17
栈 / 栈帧	按调用组织的存储 / 某次调用保存的一组状态	2、4
堆	程序动态取得和管理的内存区域	2、8
生命周期	对象从建立到不再有效的时间范围	2、8
程序 / 进程	可执行描述 / 一次运行及相关状态	2、9
内核	管理资源和保护边界的核心系统部分	3
用户态 / 内核态	不同执行权限环境，需硬件配合	3、10
系统调用	用户请求内核服务的受控接口	3、11
PID / 调度	进程编号 / 选择下一运行者的机制	3、9
虚拟地址 / 物理地址	程序使用的编号 / 翻译后的实际位置	3、7
中断 / 异常	异步事件通知 / 当前指令引发的处理事件	3、10
I/O	输入输出，连接数据提供者与接收者	3、13

术语	直接含义与回忆线索	章
ABI / 调用约定	二进制合作规则 / 参数、返回、寄存器与栈规则	4
宿主机 / 目标机器	运行工具的系统 / 工具生成代码面向的机器	5
shell / 工作目录	命令解释程序 / 相对路径的起点	5、17
交叉编译 / QEMU	面向另一目标编译 / 模拟目标硬件的程序	5
hart	RISC-V 的硬件执行线程	6
M / S / U 模式	机器、监督者、用户特权模式	6
页 / 页内偏移	固定大小地址管理单位 / 在页内的位置	7
页表 / PTE	映射结构 / 其中的一条记录	7
satp / TLB	选择页表根的控制状态 / 翻译结果缓存	7
页面错误	当前访存无法按映射和权限完成的异常	7、10
trampoline	两张页表切换前后都可执行的桥接代码	7、10
所有权 / 引用	使用与最终释放责任 / 持有对象的连接	8、13
懒分配	先登记范围，首次合法访问时才给物理页	8
trapframe / context	用户现场保存区 / 内核线程切换现场	9
RUNNABLE / RUNNING	有条件运行 / 某个 CPU 正在运行	9
SLEEPING / ZOMBIE	等待条件 / 已结束但尚待回收	9
fork / exec / wait	创建进程 / 替换内容 / 回收子进程	9、17
trap / ecall / sret	受控事件入口 / 请求入口指令 / 特权返回指令	10
临界区 / 不变量	受协调访问范围 / 正确时应保持的关系	12
原子操作 / 互斥	不可拆开的状态转换 / 一次只允许一位进入	12
自旋锁 / 睡眠锁	循环竞争的锁 / 可让出 CPU 等待的互斥锁	12
channel / wakeup	等待事件标识 / 通知对应等待者	12
丢失唤醒 / 死锁	等待与通知错过 / 执行者形成等待环	12
fd / file / off	进程槽位编号 / 打开实例 / 当前文件偏移	13

术语	直接含义与回忆线索	章
pipe / EOF	进程间内存字节管道 / 输入已结束	13
inode / 目录项	文件元数据对象 / 名字到 inode 的映射记录	14
元数据 / 位图	描述内容的信息 / 用位标记块分配情况	14
ref / nlink	内存持有引用 / 文件系统链接计数	14
buffer cache / pin	块的缓存与同步结构 / 保留对象不被回收	15
redo / 提交 / 安装	重做新状态 / 宣布可恢复 / 写回正式位置	15
幂等	同样重做多次，结果仍与做一次相同	15
驱动 / MMIO	设备协议适配代码 / 经地址访问设备寄存器	16
描述符 / ring	请求数据位置等说明 / 索引循环的队列	16
IRQ / PLIC	中断请求编号 / 外部中断控制器	16
ELF / argc / argv	装载格式 / 参数个数 / 参数字符串地址数组	17
断点 / 调用栈	指定暂停位置 / 当前函数调用关系	18
最小复现 / 回归	保留触发条件的小操作 / 修复后重测相关行为	18
COW	写时复制：先共享，写入时按协议分离	19

OPERATING SYSTEMS / XV6

附录 B 源码导航与版本差异

先按问题找到入口

想回答的问题	优先查看	章
程序怎样构建和打包	Makefile、kernel.ld、user.ld、mkfs	5
内核怎样开始执行	entry.S、start.c、main.c、userinit	6
地址怎样翻译与检查	memlayout.h、vm.c、riscv.h	7、8
物理页由谁管理	kalloc.c、uvmalloc、uvmunmap	8
进程怎样暂停和回收	proc.h、proc.c、swtch.S	9
用户怎样进入内核	trampoline.S、trap.c、kernelvec.S	10
系统调用怎样接线	user.h、usys.pl、syscall.h、syscall.c	11
锁与等待怎样配合	spinlock.c、sleeplock.c、sleep/wakeup	12
fd 与管道怎样连接	file.c、pipe.c、sysfile.c	13
路径怎样找到数据块	fs.h、fs.c、bio.c	14
故障后怎样恢复	log.c、fsinit	15
请求怎样交给设备	uart.c、plic.c、virtio_disk.c	16
命令怎样解析和装载	init.c、sh.c、exec.c、elf.h	17
哪些测试规定行为	usertests.c、forktest.c、stressfs.c	18

遇到名称不一致时怎样处理

本书按经典机制解释，开始实验前记录提交号。下表用于辨认常见演进，不能把不同版本片段直接拼成一条真实调用链。

机制	基础正文采用的形式	其他版本可能出现
第一用户程序	initcode 请求 exec init	forkret 中直接装载 init
用户返回准备	usertrapret	prepare_return 及调整后的返回路径
内核进程操作	fork、exit、wait	kfork、kexit、kwait
堆扩展	基础立即分配	懒分配、vmfault、不同 sbrk 接口
条件等待	sleep(chan, lock)	sleep_prepare 后分步释放与 sleep
内核打印	对应版本的 printf 等	printk 等名称

时钟配置也会随版本变化。读取数据结构字段时，确认它由哪条同步协议保护；不能因为名字相同就假定所有版本锁规则完全一致。

OPERATING SYSTEMS / XV6

附录 C 实验记录与进阶阅读

一份能复现的学习记录

先写源码版本、环境、CPU 数量与构建命令，再写一个待验证的具体问题。中间保留操作、预测、实际观察与差异，最后说明结论适用的范围。

例如验证 dup：预测两个 fd 共享偏移；操作是从内容 ABCD 的文件交替读取单字节；观察应是 A、B；再独立 open 对照应各从 A 开始。这个结论解释当前普通文件路径，不能直接推导所有设备都支持相同偏移行为。

若修改内核，再增加资源所有者、锁要求、失败清理和实际回归结果。一次只提出一个能观察的问题，更容易发现是哪一步理解有偏差。

继续阅读的主要入口

1. **MIT xv6-riscv 官方源码。** 阅读当前分支说明、许可证、注释与构建规则，实验时保存准确版本。https://github.com/mit-pdos/xv6-riscv
2. **xv6: a simple, Unix-like teaching operating system，2024 年版。** Russ Cox、Frans Kaashoek、Robert Morris。继续深入接口、页表、陷阱、锁、调度与文件系统。https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf
3. **MIT 6.1810，2024 课程资料。** 将实验与对应课程源码一起使用。https://pdos.csail.mit.edu/6.1810/2024/
4. **RISC-V Privileged Architecture。** 需要核对精确权限返回、陷阱与 Sv39 规则时查阅。https://docs.riscv.org/reference/isa/priv/priv-index.html
5. **RISC-V ELF psABI。** 需要核对参数寄存器、栈对齐和调用约定时查阅。https://riscv-non-isa.github.io/riscv-elf-psabi-doc/

第一次学习不必从规范全文开始。先通过一个完整例子形成问题，再到源码和规范里核对细节，通常更容易知道自己正在寻找什么。